



Lecture 1

Java SE – Programming



presentation

Java Programming – Software App Development

Assoc. Prof. Cristian Toma Ph.D.

D.I.C.E/D.E.I.C – Department of Economic Informatics & Cybernetics

www.dice.ase.ro



cristian.toma@ie.ase.ro – Business Card



Cristian Toma

IT&C Security Master

Dorobantilor Ave., No. 15-17
010572 Bucharest - Romania

<http://ism.ase.ro>

cristian.toma@ie.ase.ro

T +40 21 319 19 00 - 310

F +40 21 319 19 00



Agenda for Lecture 1 – Summary of JSE





JVM – Java Virtual Machine, Java SE – loops, methods, array

JVM & Java SE Fundamentals Intro



1.1 Java Virtual Machine Summary

Compilers & Interpreters Questions:

What is a virtual machine? How many types of JVM do you know?

What is the difference of stack based vs. registered based JVM and what are features of JIT compiler?

Should have a virtual machine associated one or more state machines?

What are advantages and disadvantages of virtual machines – native compilers vs. interpreters?

Hello World sample program in Oracle VM Box – MacOS, Linux, Windows and RaspberryPi

Command line compiling with JDK 9.0 | 8.0 – all lectures samples are command line based – and Eclipse OXYGEN projects in Linux Ubuntu 16 LTS. The sources are full in Git and alternatively is a Docker image for the lectures in the command line.
Linux Ubuntu 16 VM download:

<http://acs.ase.ro/java> ~ 15 GB HDD, 2 CPU cores, 4 GB RAM / use Oracle VM Box File->Import OVA file - All seminars will be in **/home/stud/javase/labs** directory and the lectures are in **/home/stud/javase/lectures** : **user=stud** / **pass=stud**

1.1 Java Object Oriented Programming Summary

You had graduated from lectures of Java Fundamentals – let's talk:

What is a class?

What is a package of classes in Java?

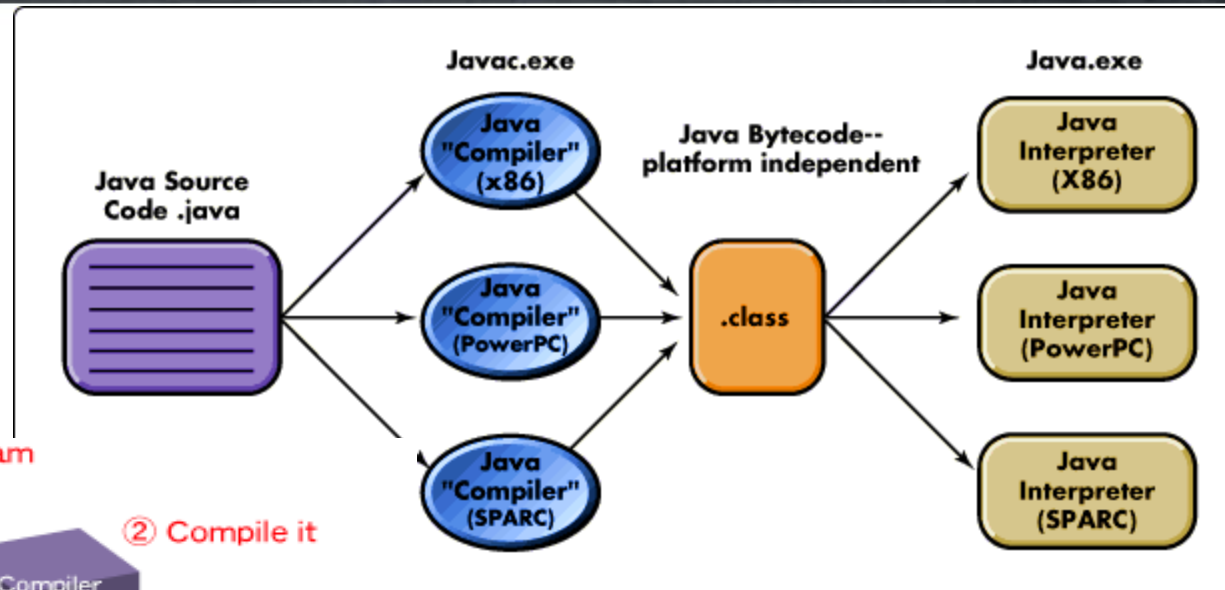
What is an object / instance?

How many bytes has an object inside JVM?

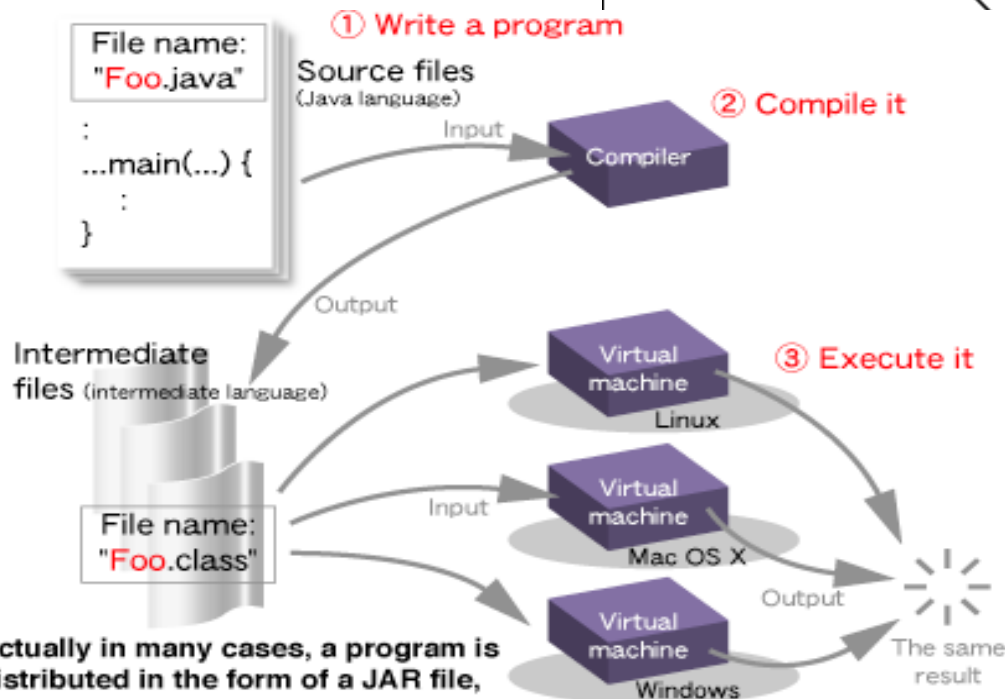
Why do we need clone, equals and hash methods?

Demo & memory model for the ***Car/Certificate*** Java class

1.1 Java Overview Summary



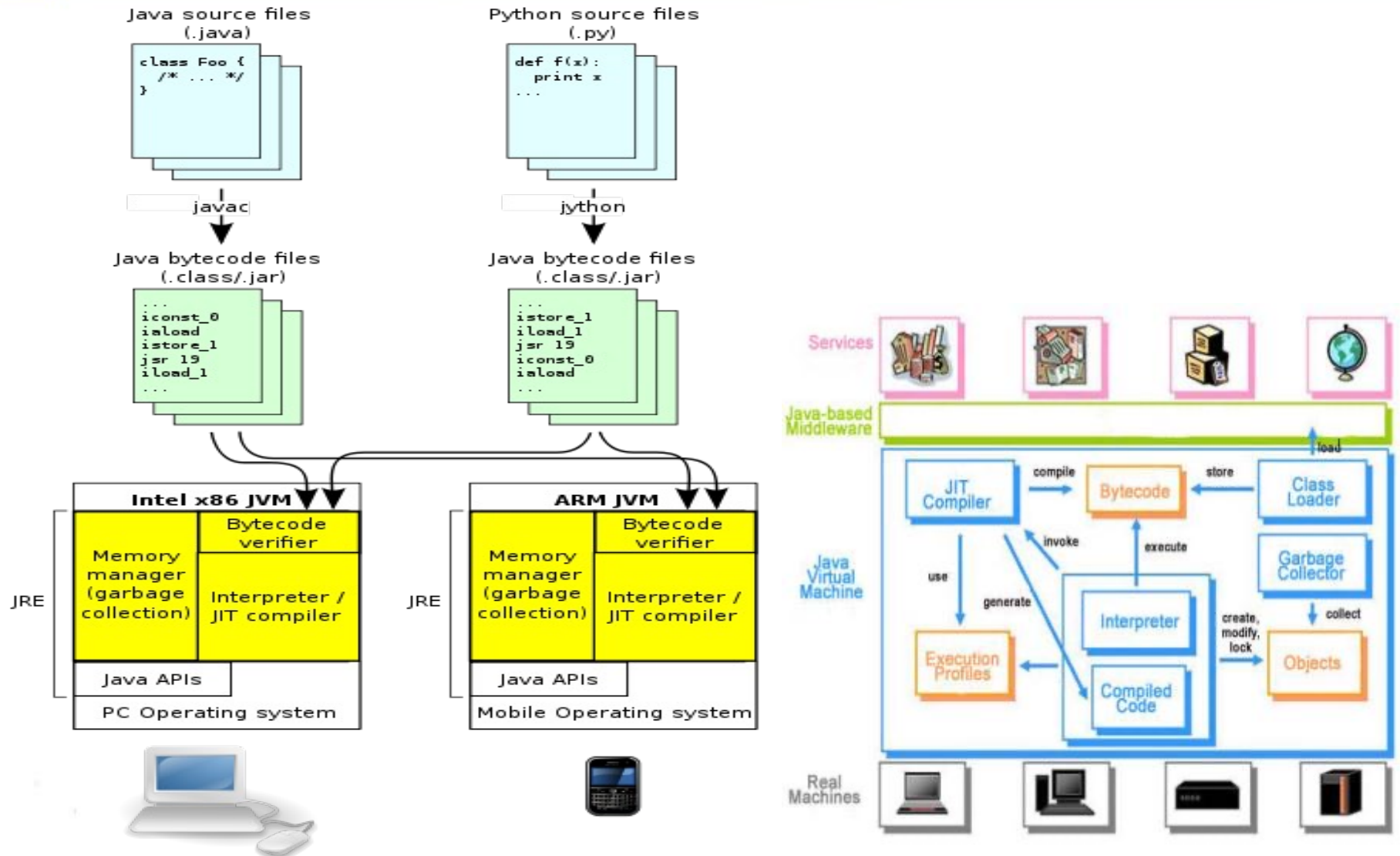
<http://support.novell.com/techcenter/articles/ana19970701.html>



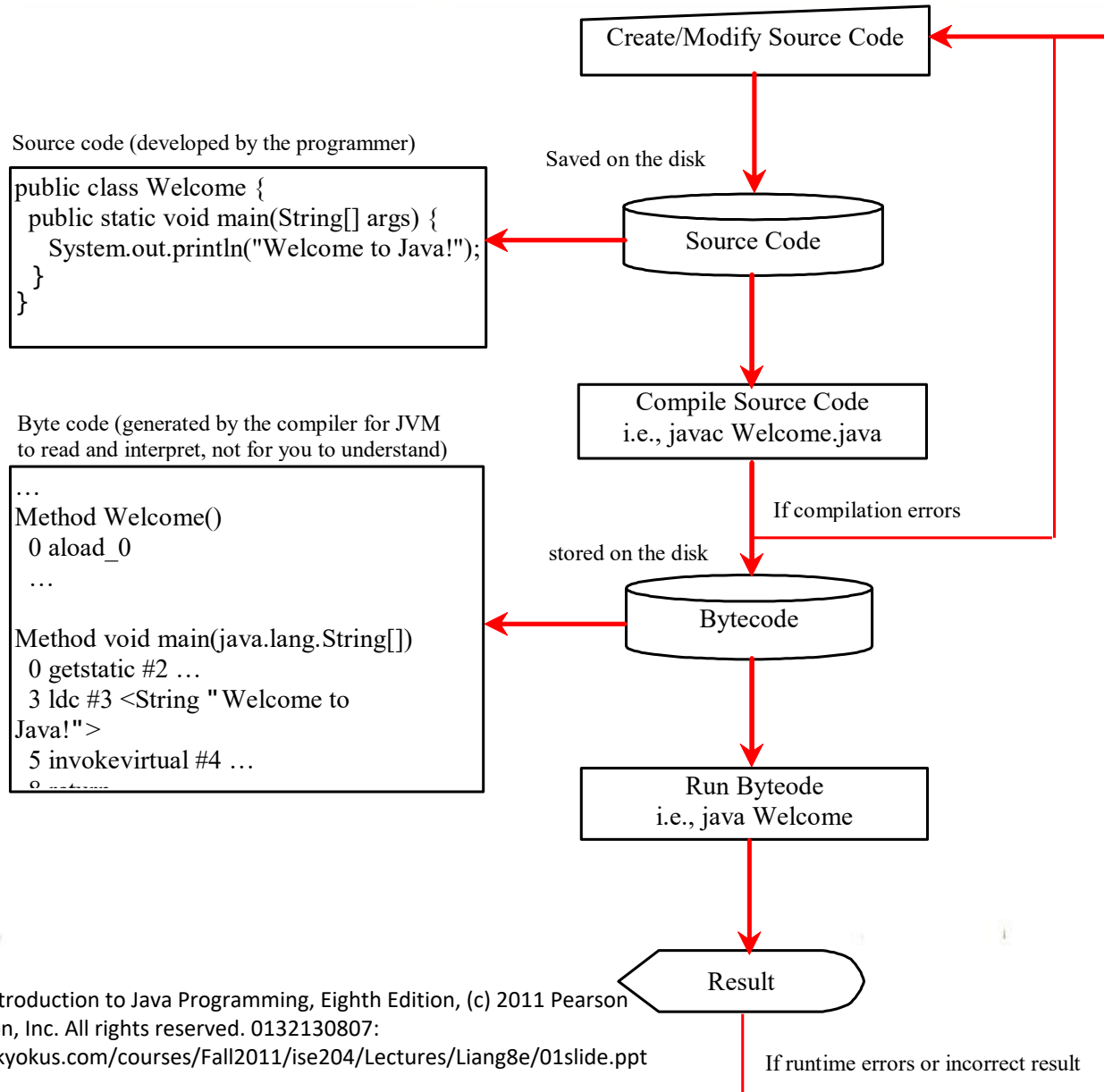
- Actually in many cases, a program is distributed in the form of a JAR file, which stands for Java ARchive file.

<http://www.media-art-online.org/java/help/how-it-works.html>

1.1 Java Virtual Machine Summary



1.1 Java Creating, Compiling, and Running Programs



DEMO

Compile in command prompt HelloWorld + show in GHex Ubuntu source and byte-code files in hex + Show Project in Eclipse + copy byte-code on Raspberry Pi and run =>

Write Once Run Anywhere

1.1 Java Trace a Program Execution

Enter main method

```
//This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

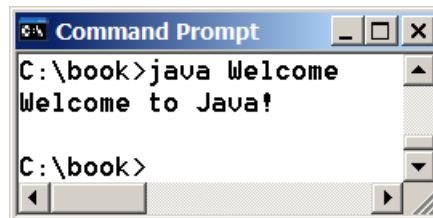
1.1 Java Trace a Program Execution

Execute statement

```
//This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

1.1 Java Trace a Program Execution

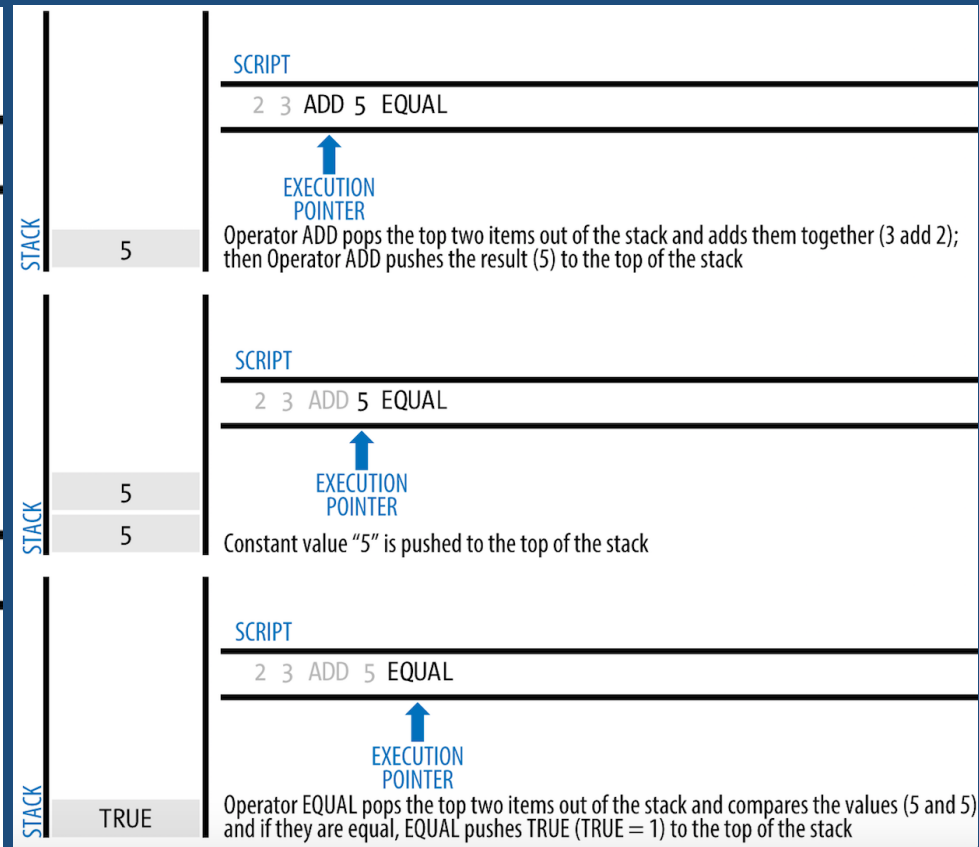
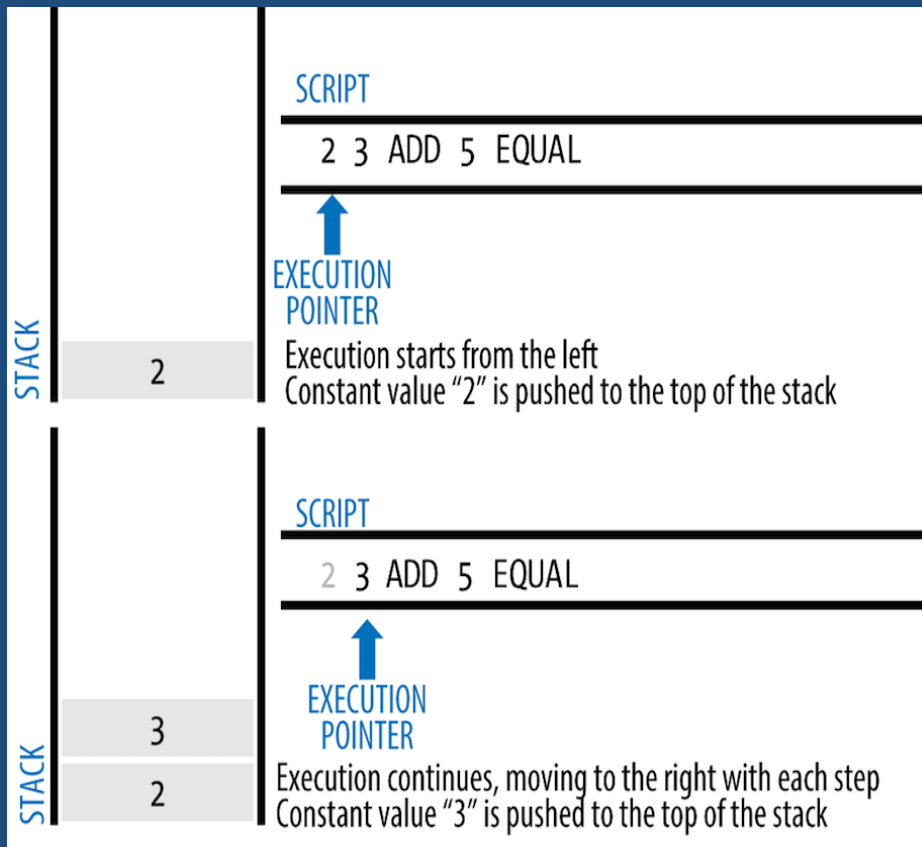
```
//This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```



```
Command Prompt  
C:\book>java Welcome  
Welcome to Java!  
C:\book>
```

print a message to the console

* Java VM ⇔ Bitcoin / Ethereum Blockchain Technology - Transactions



Copyright: :Andreas M. Antonopoulos. "Mastering Bitcoin, 2nd Edition", Publisher: O'Reilly Media, Inc.,
Release Date: June 2017, ISBN: 9781491954379

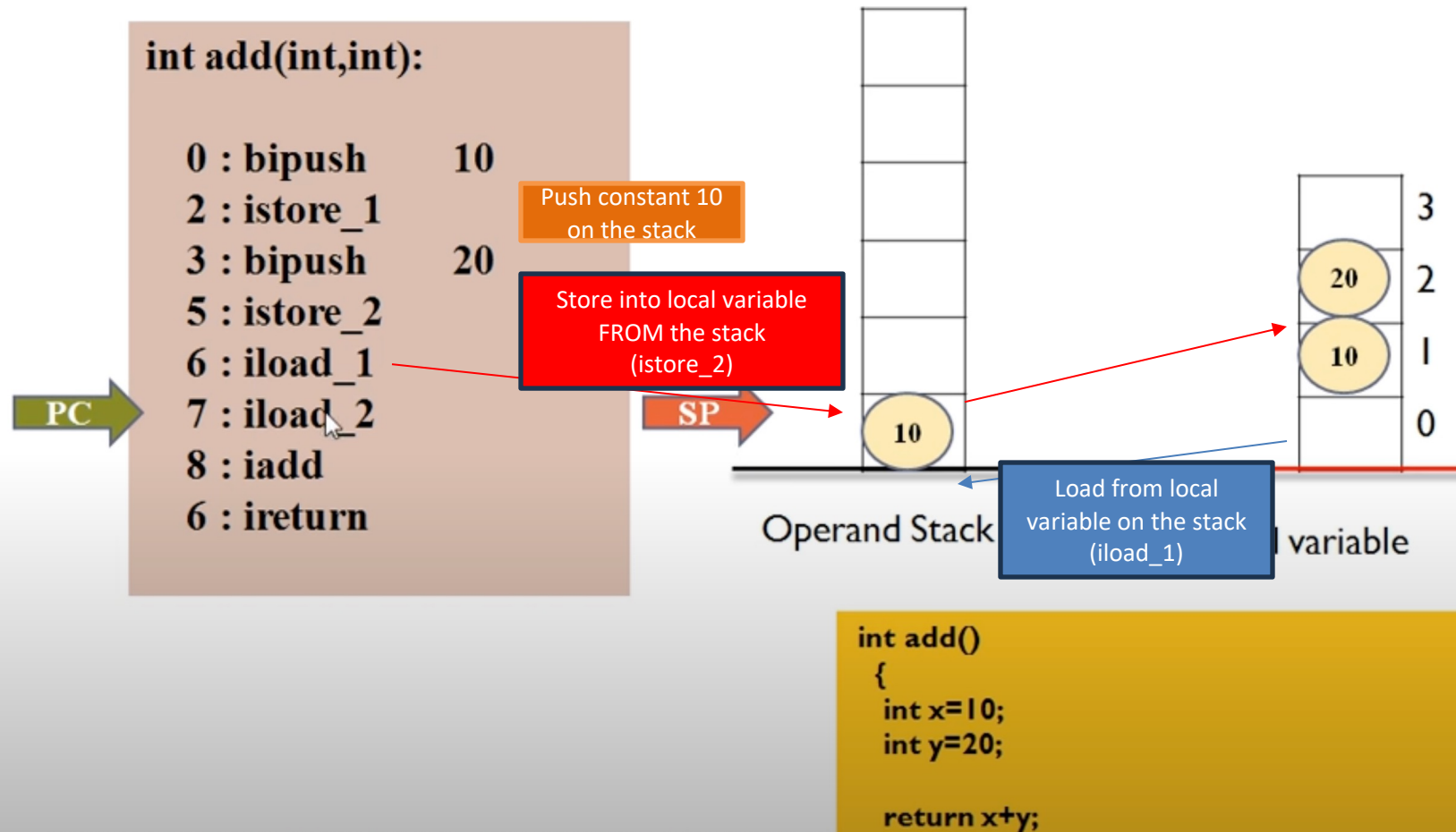
Oracle Java Bytecode Crash Course: <https://www.youtube.com/watch?v=e2zmmkc5xl0>
<https://docs.oracle.com/javase/specs/jvms/se21/html/jvms-2.html>

1.1 Java Bytecode investigation

https://en.wikipedia.org/wiki/List_of_Java_bytecode_instructions

<https://stackoverflow.com/questions/34559067/java-bytecode-beginner-questions-order-of-istore-variable-byte-usage>

Java program



<https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html> | https://en.wikipedia.org/wiki/Java_class_file | https://en.wikipedia.org/wiki/List_of_Java_bytecode_instructions

<https://www.baeldung.com/java-asm> | <https://asm.ow2.io/> | <https://tomassetti.me/getting-started-with-javaparser-analyzing-java-code-programmatically/>

<https://www.infoworld.com/article/2076954/how-to-build-an-interpreter-in-java--part-2--the-structure.html>
<https://www.infoworld.com/article/2076974/build-an-interpreter-in-java----implement-the-execution-engine.html?page=2>

1.1 Java Bytecode investigation

https://en.wikipedia.org/wiki/List_of_Java_bytecode_instructions

<https://stackoverflow.com/questions/34559067/java-bytecode-beginner-questions-order-of-istore-variable-usage-byte-usage>

```
% pwd
/Users/critoma/eclipse-workspace-2023/c001

% ls eu/ase/hello/
HelloWorld.java

% cat eu/ase/hello/HelloWorld.java
package eu.ase.hello;

public class HelloWorld {
    public static void main(String[] argv)
    {
        int r = 0;
        int a = 9;
        int b = -2;
        r = a + b;
        System.out.println("Hello World form Java!!! r = " + r);
    } // end main

% javac eu/ase/hello/HelloWorld.java

% ls eu/ase/hello/
HelloWorld.class HelloWorld.java

% java eu.ase.hello.HelloWorld
Hello World form Java!!! r = 7
```

```
% javap -c -p eu/ase/hello/HelloWorld.class
Compiled from "HelloWorld.java"
public class eu.ase.hello.HelloWorld {
    public eu.ase.hello.HelloWorld();
        Code:
            0: aload_0
            1: invokespecial #1                      //
Method java/lang/Object."<init>":()V
            4: return
    public static void main(java.lang.String[]);
        Code:
            0: iconst_0    // (03) load the int value 0 onto the stack
            1: istore_1    // (3C) store int value into variable 1 (r) from stack
            2: bipush 9    // (10 09)
                // push a byte onto the stack as an integer value
            4: istore_2    // (3D) store int value into variable 2 (a) from stack
            5: bipush -2   // (10 FE)
                // push a byte onto the stack as an integer value
            7: istore_3    // (3E) store int value into variable 3 (b) from stack
            8: iload_2     // (1C) load an int value from
                // local variable 2 (a) on to the stack
            9: iload_3     // (1D) load an int value
                // from local variable 3 (b) on to the stack
            10: iadd        // (60) add two ints (operate only on STACK)
                // pop 2 values from the stack add them
                // in CPU and store the result back on the
                // STACK
            11: istore_1    // (3C) store int value from the STACK
                // into variable 1 (r) in local variable
            12: getstatic   #7                      // Field
java/lang/System.out:Ljava/io/PrintStream;
            15: iload_1
            16: invokedynamic #13, 0                  // InvokeDynamic
#0:makeConcatWithConstants:(I)Ljava/lang/String;
            21: invokevirtual #17                    // Method
java/io/PrintStream.println:(Ljava/lang/String;)V
            24: return
    }
% xxd eu/ase/hello/HelloWorld.class | less
```

1.1 Java Bytecode investigation

https://en.wikipedia.org/wiki/List_of_Java_bytecode_instructions

<https://stackoverflow.com/questions/34559067/java-bytecode-beginner-questions-order-of-istore-variable-usage-byte-usage>

```
00000160: 6c6c 6f57 6f72 6c64 2e6a 6176 6101 0010 HelloWorld.java:bytecode
00000170: 426f 6f74 7374 7261 704d 6574 686f 6473 BootstrapMethods
00000180: 0800 2101 001e 4865 6c6c 6f20 576f 726c ..!...Hello Worl
00000190: 6420 666f 726d 204a 6176 6121 2121 2072 d form Java!!! r
000001a0: 203d 2001 0f06 0023 0a00 2400 2507 0026 = ....#...$.%..&
000001b0: 0c00 0f00 2701 0024 6a01 7661 2f6c 616e ....'$.java/lan
000001c0: 672f 696e 766f 6b65 2f53 7472 696e 6743 g/invoke/StringC
000001d0: 6f6e 6361 7446 6163 746f 7279 0100 9828 oncatFactory...(
000001e0: 4c6a 6176 612f 6c61 6e67 2f69 6e76 6f6b Ljava/lang/invo
000001f0: 652f 4d65 7468 6f64 4861 6e64 6c65 7324 e/MethodHandles$
00000200: 4c6f 6f6b 7570 3b4c 6a61 7661 2f6c 616e Lookup;Ljava/lan
00000210: 672f 5374 7269 6e67 3b4c 6a61 7661 2f6c g/String;Ljava/l
00000220: 616e 672f 696e 766f 6b65 2f4d 6574 686f ang/invoke/Metho
00000230: 6454 7970 653b 4c6a 6176 612f 6c61 6e67 dType;Ljava/lang
00000240: 2f53 7472 696e 673b 5b4c 6a61 7661 2f6c /String;[Ljava/l
00000250: 616e 672f 4f62 6a65 6374 3b29 4c6a 6176 ang/Object;Ljav
00000260: 612f 6c61 6e67 2f69 6e76 6f6b 652f 4361 a/lang/invoke/Ca
00000270: 6c6c 5369 7465 3b01 000c 496e 6e65 7243 llSite;...InnerC
00000280: 6c61 7373 6573 0700 2a01 0025 6a61 7661 lasses...*..%java
00000290: 2f6c 616e 672f 696e 766f 6b65 2f4d 6574 /lang/invoke/Met
000002a0: 686f 6448 616e 646c 6573 244c 6f6f 6b75 hodHandles$Looku
000002b0: 7007 002c 0100 1e6a 6176 612f 6c61 6e67 p,...java/lang
000002c0: 2f69 6e76 6f6b 652f 4d65 7468 6f64 4861 /invoke/MethodHa
000002d0: 6e64 6c65 7301 0006 4c6f 6f6b 7570 0021 ndles...Lookup.!
000002e0: 0017 0002 0000 0000 0002 0001 0005 0006 .....
000002f0: 0001 0019 0000 001d 0001 0001 0000 0005 .....
00000300: 2ab7 0001 b100 0000 0100 1a00 0000 0600 *.....
00000310: 0100 0000 0300 0900 1b00 1c00 0100 1900 .....
00000320: 0000 3d00 0200 0400 0000 1903 3c10 093d ..=.....<..=
00000330: 10fe 3e1c 1d60 3cb2 0007 1bba 000d 0000 ..>..<.....
00000340: b600 11b1 0000 0001 001a 0000 0012 0004 .....
00000350: 0000 0005 0008 0006 000c 0007 0018 0008 .....
00000360: 0003 001d 0000 0002 001e 001f 0000 0008 .....
00000370: 0001 0022 0001 0020 0028 0000 000a 0001 ..."...(.
00000380: 0029 002b 002d 0019 .....
(END)
```

```
% javap -c -p eu/ase/hello/HelloWorld.class
```

```
Compiled from "HelloWorld.java"
```

```
public class eu.ase.hello.HelloWorld {
```

```
    public eu.ase.hello.HelloWorld();
```

```
        Code:
```

```
            0: aload_0
```

```
            1: invokespecial #1
```

```
                // Method
```

```
java/lang/Object."<init>":()V
```

```
            4: return
```

```
    public static void main(java.lang.String[]);
```

```
        Code:
```

```
            0: iconst_0    // (03) load the int value 0 onto the stack
```

```
            1: istore_1    // (3C) store int value into variable 1 (r) from stack
```

```
            2: bipush 9    // (10 09)
```

```
                // push a byte onto the stack as an integer value
```

```
            4: istore_2    // (3D) store int value into variable 2 (a) from stack
```

```
            5: bipush -2   // (10 FE)
```

```
                // push a byte onto the stack as an integer value
```

```
            7: istore_3    // (3E) store int value into variable 3 (b) from stack
```

```
            8: iload_2     // (1C) load an int value from
```

```
                // local variable 2 (a) on to the stack
```

```
            9: iload_3     // (1D) load an int value
```

```
                // from local variable 3 (b) on to the stack
```

```
           10: iadd        // (60) add two ints (operate only on STACK)
```

```
                // pop 2 values from the stack add them
```

```
                // in CPU and store the result back on the
```

```
                // STACK
```

```
           11: istore_1    // (3C) store int value from the STACK
```

```
                // into variable 1 (r) in local variable
```

```
           12: getstatic   #7
```

```
                // Field
```

```
java/lang/System.out:Ljava/io/PrintStream;
```

```
           15: iload_1
```

```
           16: invokedynamic #13, 0
```

```
                // InvokeDynamic
```

```
#0:makeConcatWithConstants:(I)Ljava/lang/String;
```

```
           21: invokevirtual #17
```

```
                // Method
```

```
java/io/PrintStream.println:(Ljava/lang/String;)V
```

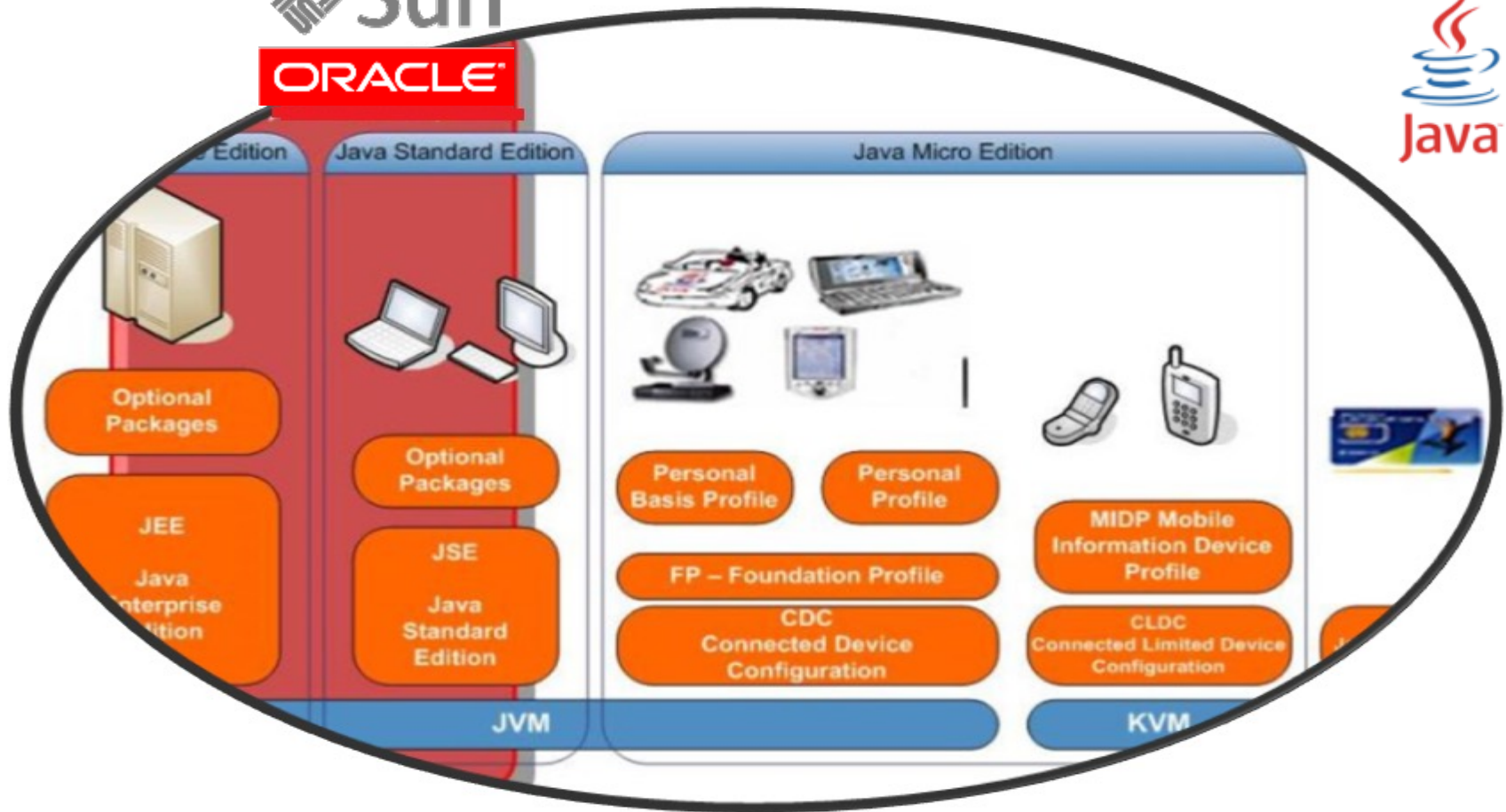
```
           24: return
```

```
}
% xxd eu/ase/hello/HelloWorld.class | less
```

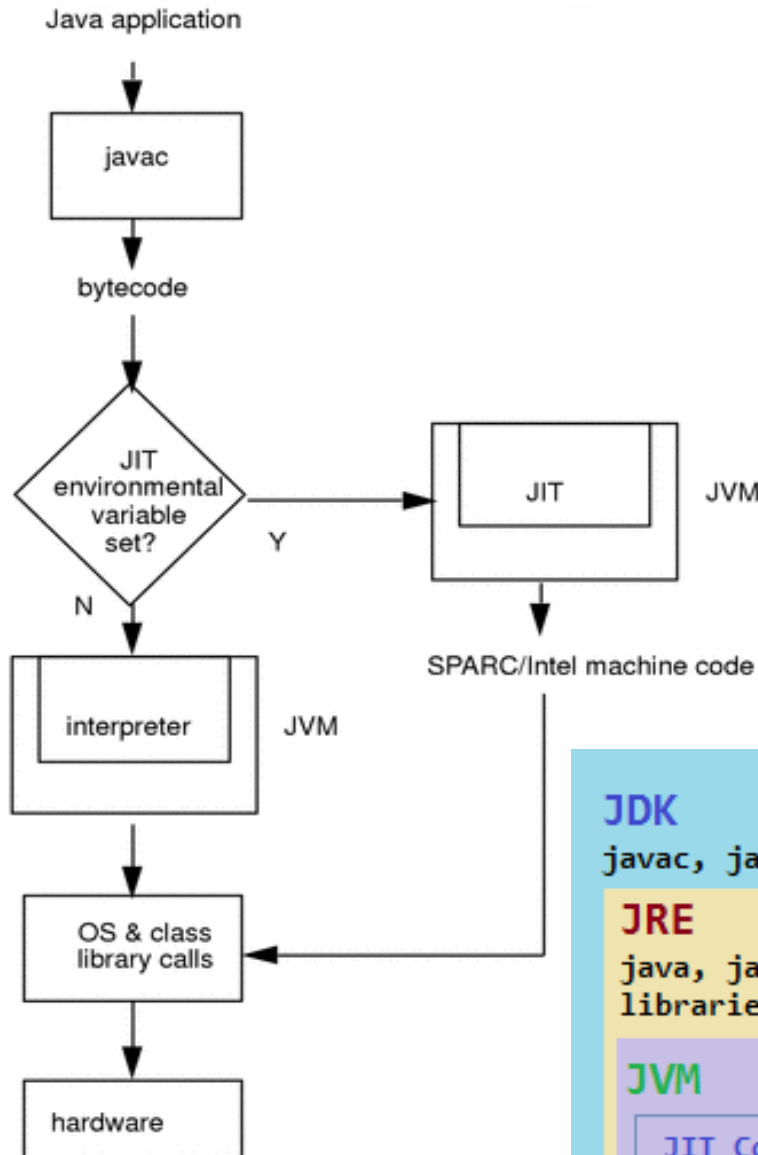
Explain compile (javac) and interpret (java) plus byte-code in video:

<https://www.youtube.com/watch?v=d86ws7mQYlg> (min 00:52 – 01:05)

JSE – Java Standard Edition



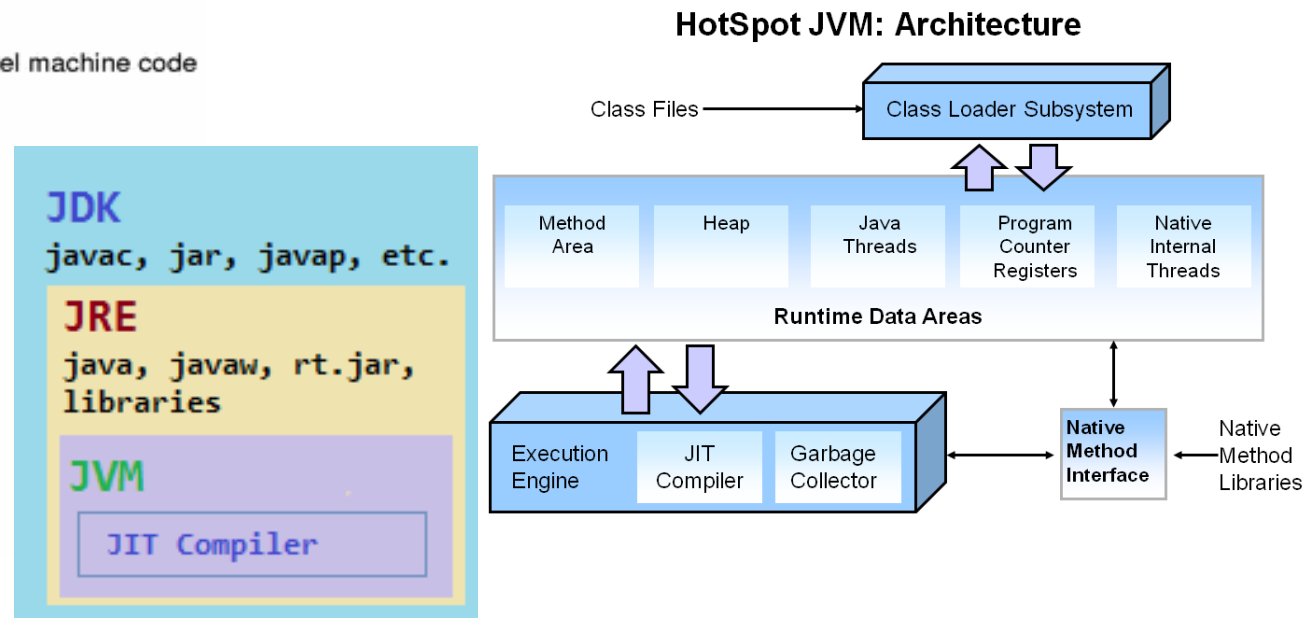
1.1 Java SE JDK and JRE



JDK = JRE + Development / Compiling / Debugging tools

JRE = JVM + Java Packages Classes (like util, math, lang, awt etc) + Runtime libraries.

JVM = Class loader system + Runtime data area + Execution Engine.



1.1 Java VM Architecture

* The former Java 8 - \$JAVA_HOME/lib/rt.jar - is split into multiple modules starting
with Java 9 - JDK 9, ... 11, ... 17, 19, 21 ...

```
% /Users/critoma/Desktop/PhH20230505/software/zulu8.68.0.21-ca-jdk8.0.362-  
macosx_aarch64/zulu-8.jdk/Contents/Home/bin/jar tf  
/Users/critoma/Desktop/PhH20230505/software/zulu8.68.0.21-ca-jdk8.0.362-  
macosx_aarch64/zulu-8.jdk/Contents/Home/jre/lib/rt.jar | grep java/lang/System  
java/lang/System$1.class  
java/lang/SystemClassLoaderAction.class  
java/lang/System$2.class  
java/lang/System.class
```

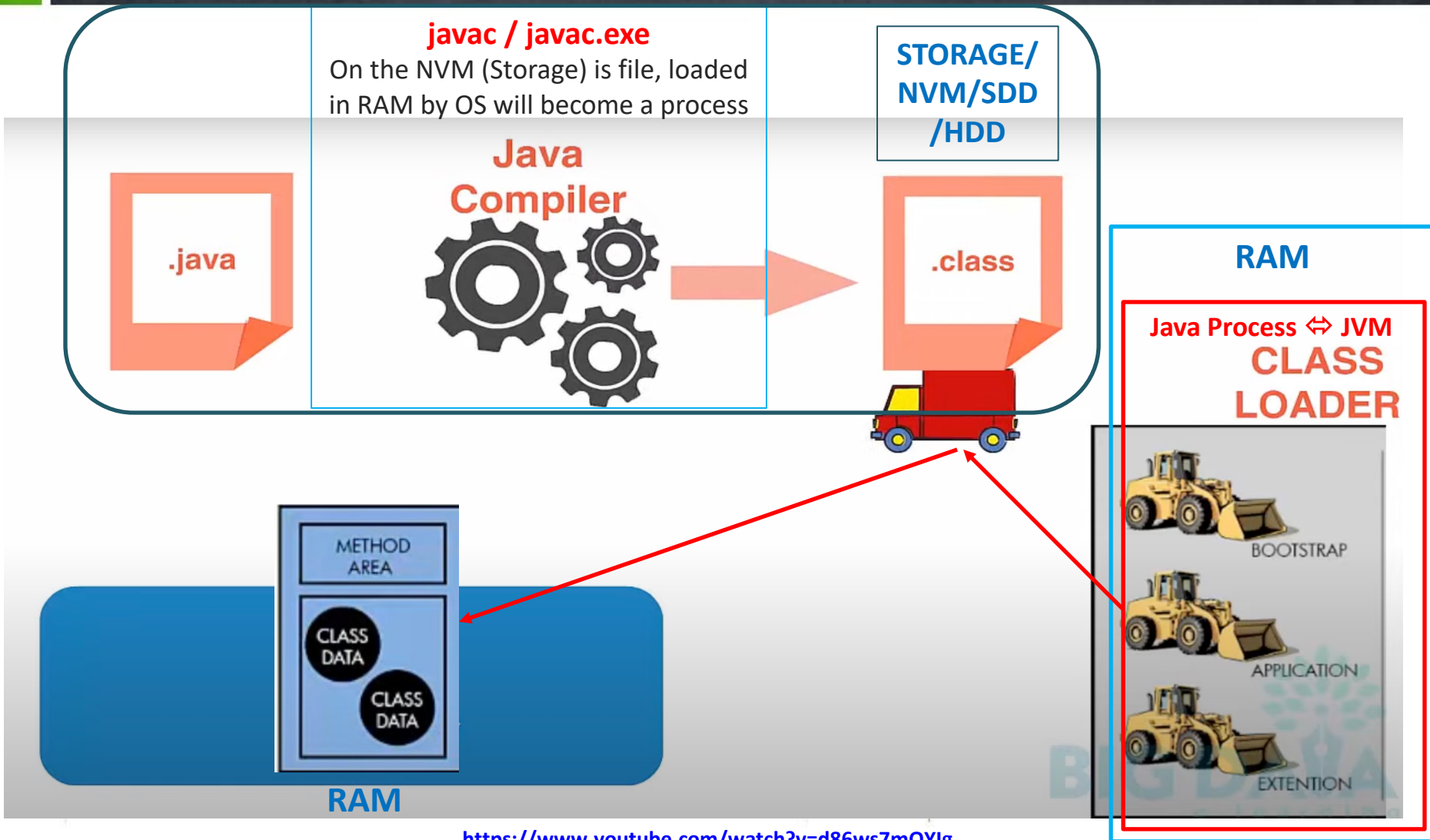
```
% /Library/Java/JavaVirtualMachines/jdk-21.0.1.jdk/Contents/Home/bin/jmod list  
/Library/Java/JavaVirtualMachines/jdk-21.0.1.jdk/Contents/Home/jmods/java.base.jmod |  
grep java/lang/System
```

```
classes/java/lang/System$1.class  
classes/java/lang/System$2.class  
classes/java/lang/System$CallersHolder.class  
classes/java/lang/System$Logger$Level.class  
classes/java/lang/System$Logger.class  
classes/java/lang/System$LoggerFinder.class  
classes/java/lang/System.class
```

Reverse Engineering - Decompiled

```
25 * Copyright (c) 1994, 2023, Oracle and/or its affiliates.  
26 package java.lang;  
27 import java.io.BufferedReader;  
28 import java.io.BufferedOutputStream;  
29 import java.io.Console;  
30 import java.io.FileDescriptor;  
31 import java.io.FileInputStream;  
32 import java.io.FileOutputStream;  
33 import java.io.IOException;  
34 import java.io.InputStream;  
35 import java.io.OutputStream;  
  
111 public final class System {  
112     /* Register the natives via the static initializer.  
113     *  
114     * The VM will invoke the initPhase1 method to complete the initialization  
115     * of this class separate from <clinit>.  
116     */  
117     private static native void registerNatives();  
118     static {  
119         registerNatives();  
120     }  
121  
122     /** Don't let anyone instantiate this class */  
123     private System() {  
124     }  
125  
127     * The "standard" input stream. This stream is already  
128     public static final InputStream in = null;  
129  
141     * The "standard" output stream. This stream is already  
170     public static final PrintStream out = null;
```

1.1 Java VM Architecture



<https://www.youtube.com/watch?v=d86ws7mQYlg>

<https://www.youtube.com/watch?v=b6MxozZv6-M>

<https://www.youtube.com/playlist?list=PL1bEcqKguUdOHI1nqggWCqTWx9vyoyWJ9>

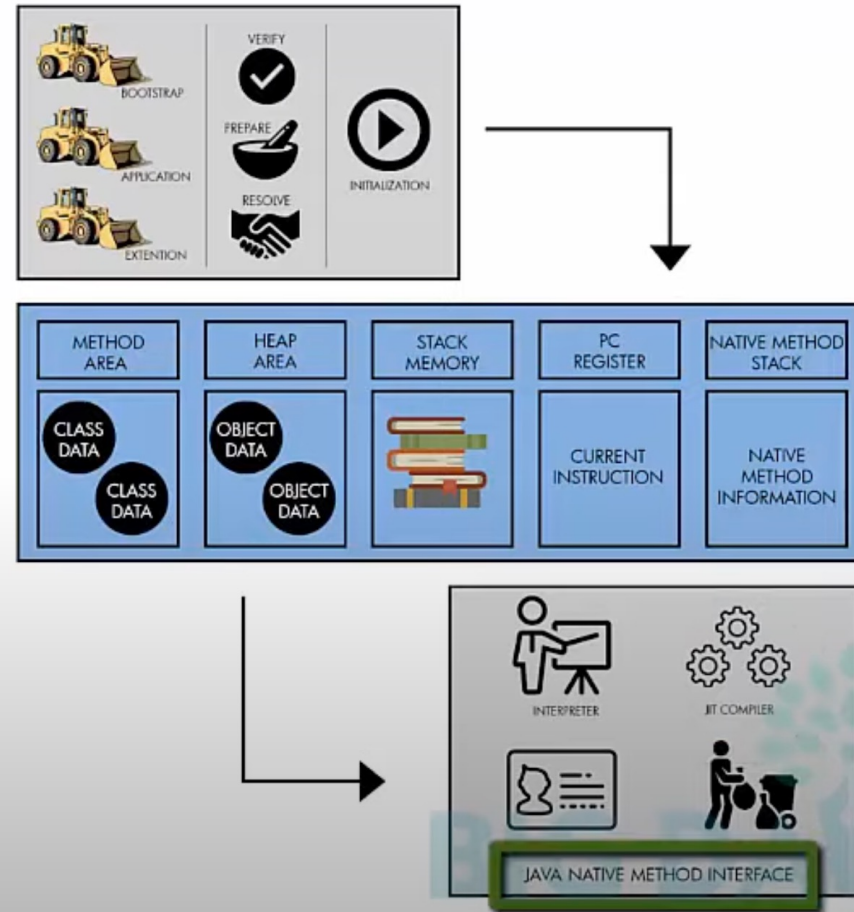
<https://www.youtube.com/playlist?list=PLRJP-g0nSX0e0kgIW6bom0kQtx4dyHyZx>

<https://www.youtube.com/watch?v=QHIWkwxs0AI>

1.1 Java VM Architecture

JAVA PROCESS in RAM ⇔ JVM

- Class loader
loading phase,
linking phase, and
Initialization phase.
- Run time data area
method area,
heap memory,
stack memory,
PC registers &
JNI.
- Execution engine
Interpreter,
JIT compiler,
JNI, and
Garbage collector.



Abstract Model Memory Layout in RAM

eclipse-workspace-jse - c01/src/c01/ProgMainCertificate.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Debug Project Explorer

ProgMainCertificate [Java Application]
c01.ProgMainCertificate at localhost:61762
Thread [main] (Suspended)
ProgMainCertificate.main(String[]) line: 29
D:\PROGRAMS\eclipse\plugins\org.eclipse.justi.openjdk.hotspot

Certificate.java ProgMainCertificate.java

```
14 c2 = c1;  
15 //c2 = c1.myClone();  
16 c2.setIdCert(333);  
17  
18 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
19 System.out.println("c2 - id = " + c2.getIdCert() + ", iName = " + c2.getIssuerName());  
20  
21  
22 Certificate c3 = new Certificate(222, "C3");  
23 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
24 System.out.println("c3 - id = " + c3.getIdCert() + ", iName = " + c3.getIssuerName());  
25  
26 c3 = c1.myClone();  
27 c3.setIdCert(111);  
28  
29 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
30 System.out.println("c3 - id = " + c3.getIdCert() + ", iName = " + c3.getIssuerName());  
31  
32 }  
33 }
```

Variables

Name	Value
setIdCert()	(No explicit return value)
args	String[] (id=20)
c1	Certificate (id=21)
idCert	333
issuerName	"Verisign" (id=26)
c2	Certificate (id=21)
c3	Certificate (id=53)
idCert	111
issuerName	"Verisign" (id=26)

Breakpoints Expressions

<Choose a previously entered expression>

@26 2. @32 len value

Console Problems Debug Shell

ProgMainCertificate [Java Application] D:\PROGRAMS\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_15.0.1.v20201027-0507\jre\bin\javaw.exe (Mar 2, 2021, 12:05:13 PM)

```
c1 - id = 777, iName = Verisign  
c2 - id = 999, iName = GoDaddy  
c1 - id = 333, iName = Verisign  
c2 - id = 333, iName = Verisign  
c1 - id = 333, iName = Verisign  
c3 - id = 222, iName = C3
```

heap

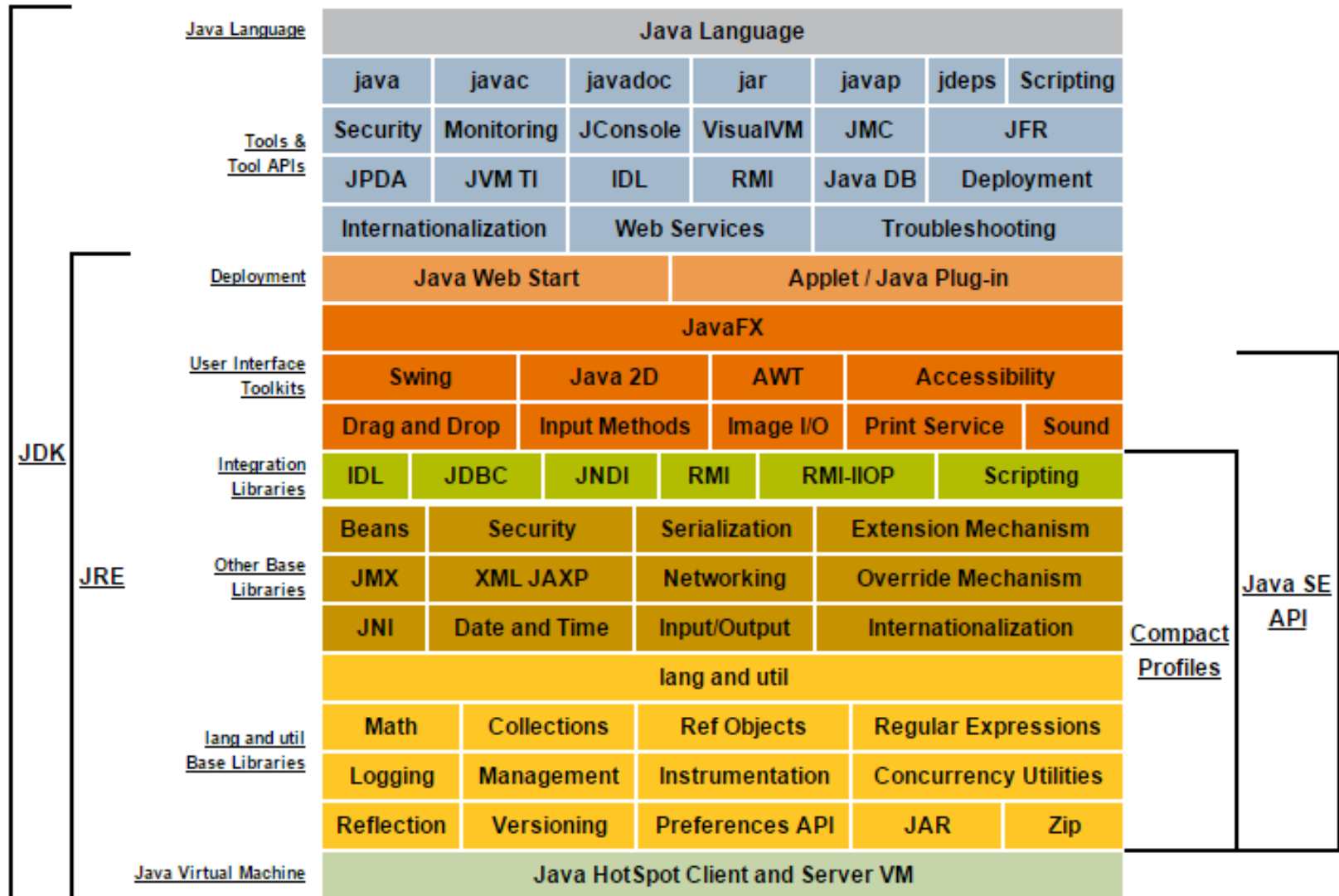
stack

byte-code

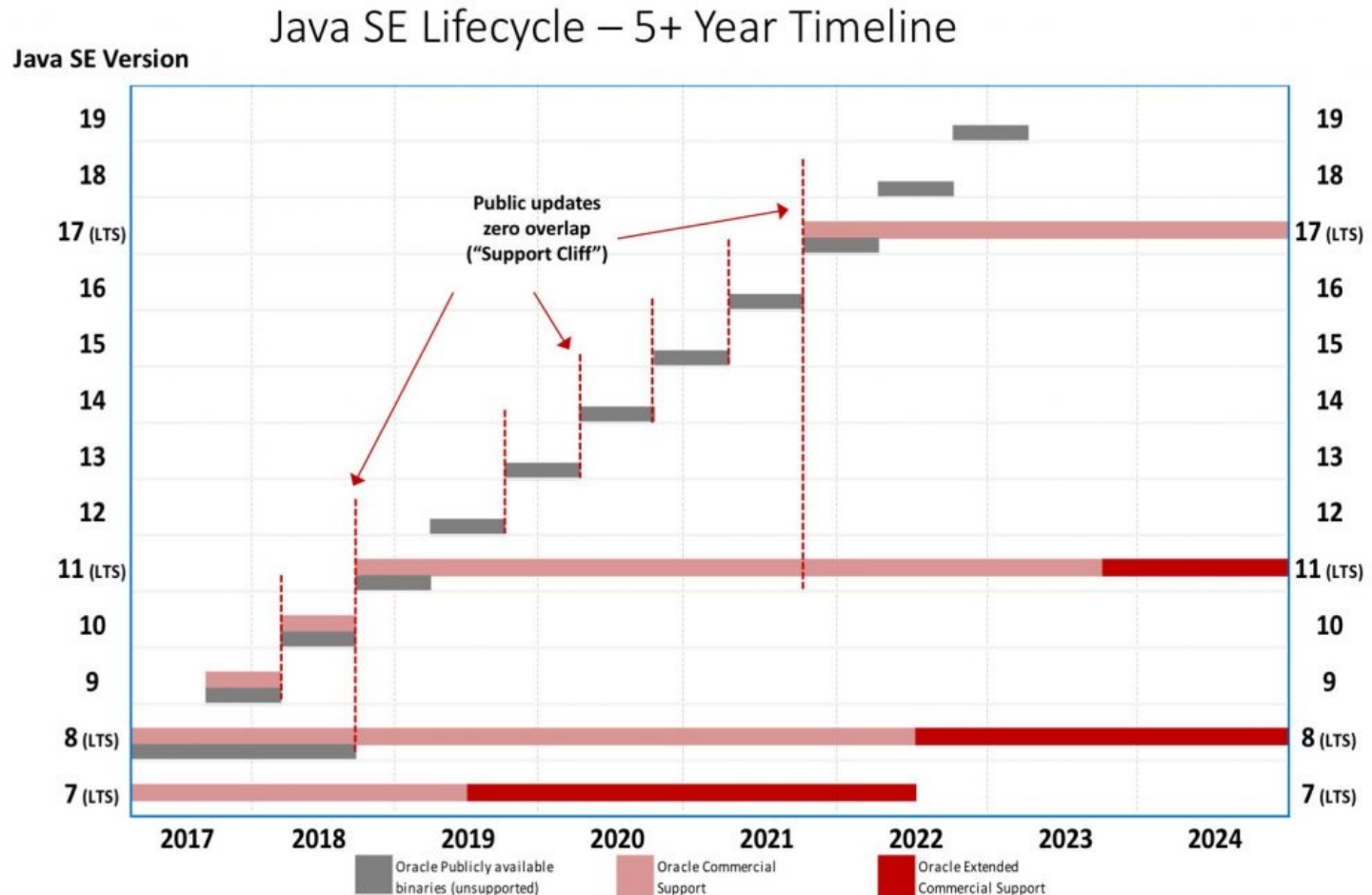
myClone() Certificate main

1.1 Java Development Kit 7/8 Diagram

Description of Java Conceptual Diagram



1.1 Java Development Process



Examples of the JDKs Java SE implementations:

1. Free SDK and Commercial Support – Oracle OTN:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

2. Community Free SDK and Open Source – OpenJDK:

<http://openjdk.java.net/> | [JDK Enhancement Proposals – JEP for Java Community: http://openjdk.java.net/jeps/0](#)

1.2 Java Selections – Boolean Operators, if-else, switch statements

The `boolean` Type and Operators

Often in a program you need to compare two values, such as whether `i` is greater than `j`. Java provides six comparison operators (also known as relational operators) that can be used to compare two values. The result of the comparison is a Boolean value: `true` or `false`.

```
boolean b = (1 > 2);
```

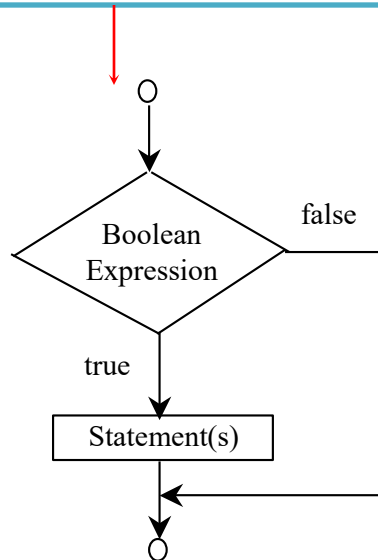
Comparison Operators

<i>Operator</i>	<i>Name</i>
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	equal to
!=	not equal to

1.2 Java Selections – Boolean Operators, if-else, switch statements

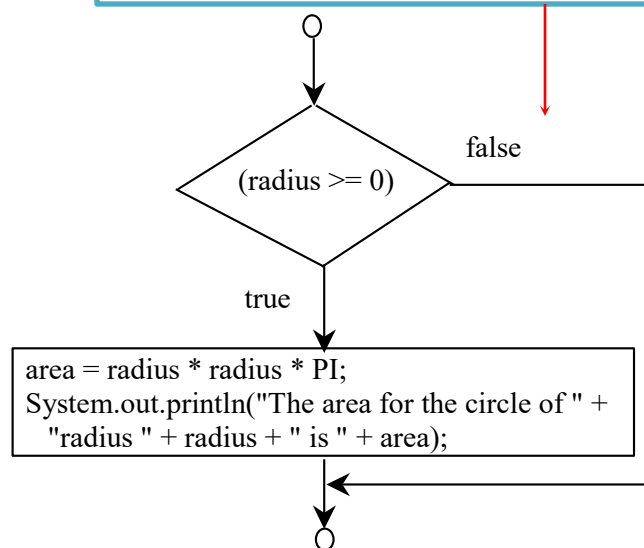
One-way if Statements

```
if (boolean-expression) {  
    statement(s);  
}
```



(A)

```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area"  
        + " for the circle of radius "  
        + radius + " is " + area);  
}
```



(B)

1.2 Java Selections – Boolean Operators, if-else, switch statements

Note

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

(a) Wrong

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(b) Correct

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(a)

Equivalent

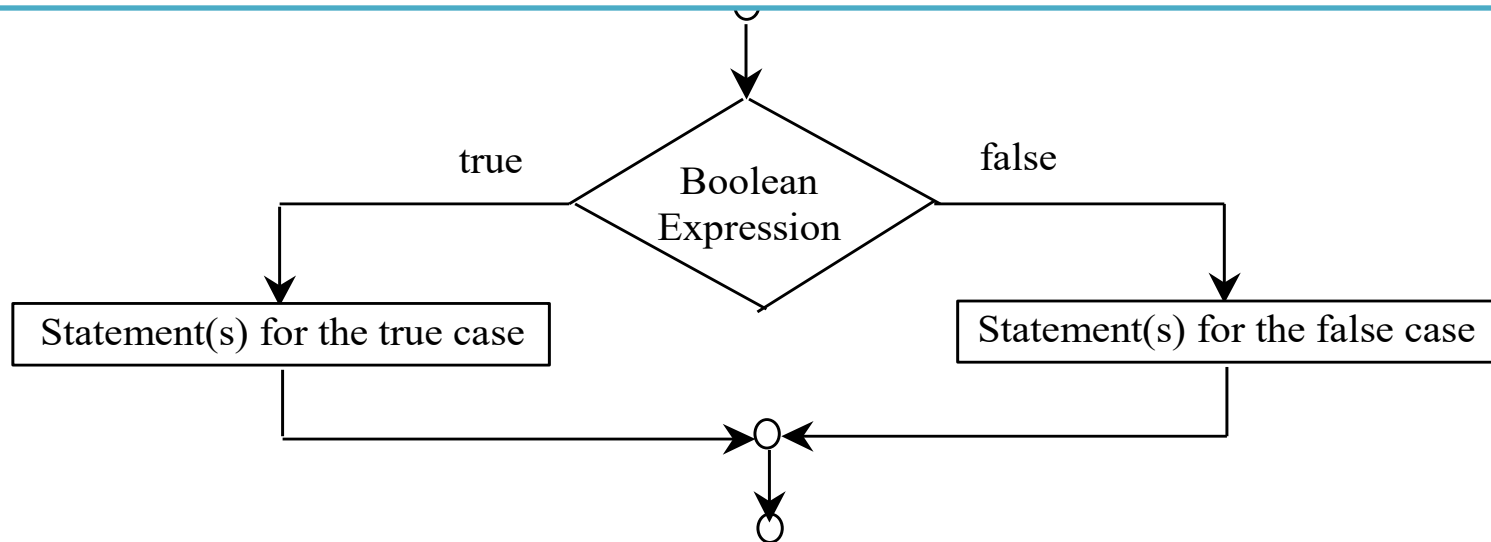
```
if (i > 0)  
    System.out.println("i is positive");
```

(b)

1.2 Java Selections – Boolean Operators, if-else, switch statements

The Two-way `if` Statement

```
if (boolean-expression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```



1.2 Java Selections – Boolean Operators, if-else, switch statements

if...else Example

```
if (radius >= 0) {  
    area = radius * radius * 3.14159;  
  
    System.out.println("The area for the "  
        + "circle of radius " + radius +  
        " is " + area);  
}  
else {  
    System.out.println("Negative input");  
}
```

Multiple Alternative if Statements

```
if (score >= 90.0)
    grade = 'A';
else
    if (score >= 80.0)
        grade = 'B';
    else
        if (score >= 70.0)
            grade = 'C';
        else
            if (score >= 60.0)
                grade = 'D';
            else
                grade = 'F';
```

Equivalent

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace if-else statement

Suppose score is 70.0

The condition is false

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace if-else statement

Suppose score is 70.0

The condition is true

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace if-else statement

Suppose score is 70.0

grade is C

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace if-else statement

Suppose score is 70.0

Exit the if statement

```
if (score >= 90.0)
    grade = 'A';
else if (score >= 80.0)
    grade = 'B';
else if (score >= 70.0)
    grade = 'C';
else if (score >= 60.0)
    grade = 'D';
else
    grade = 'F';
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Note

The else clause matches the most recent if clause in the same block.

```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
    if (i > k)
        System.out.println("A");
else
    System.out.println("B");
```

(a)

Equivalent

```
int i = 1;
int j = 2;
int k = 3;

if (i > j)
    if (i > k)
        System.out.println("A");
else
    System.out.println("B");
```

(b)

1.2 Java Selections – Boolean Operators, if-else, switch statements

Note, cont.

Nothing is printed from the preceding statement. To force the else clause to match the first if clause, you must add a pair of braces:

```
int i = 1;
int j = 2;
int k = 3;
if (i > j) {
    if (i > k)
        System.out.println("A");
}
else
    System.out.println("B");
```

This statement prints B.

1.2 Java Selections – Boolean Operators, if-else, switch statements

Common Errors

Adding a semicolon at the end of an if clause is a common mistake.

```
if (radius >= 0);
```



Wrong

```
{  
    area = radius*radius*PI;  
    System.out.println(  
        "The area for the circle of radius " +  
        radius + " is " + area);  
}
```

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.

This error often occurs when you use the next-line block style.

1.2 Java Selections – Boolean Operators, if-else, switch statements

TIP

```
if (number % 2 == 0)
    even = true;
else
    even = false;
```

(a)

Equivalent

```
boolean even
    = number % 2 == 0;
```

(b)

CAUTION

```
if (even == true)
    System.out.println(
        "It is even.");
```

(a)

Equivalent

```
if (even)
    System.out.println(
        "It is even.");
```

(b)

1.2 Java Selections – Boolean Operators, if-else, switch statements

Problem: Computing Taxes

The US federal personal income tax is calculated based on the filing status and taxable income. There are four filing statuses: single filers, married filing jointly, married filing separately, and head of household. The tax rates for 2009 (2014?) are shown below.

Marginal Tax Rate	Single	Married Filing Jointly or Qualified Widow(er)	Married Filing Separately	Head of Household
10%	\$0 – \$8,350	\$0 – \$16,700	\$0 – \$8,350	\$0 – \$11,950
15%	\$8,351 – \$33,950	\$16,701 – \$67,900	\$8,351 – \$33,950	\$11,951 – \$45,500
25%	\$33,951 – \$82,250	\$67,901 – \$137,050	\$33,951 – \$68,525	\$45,501 – \$117,450
28%	\$82,251 – \$171,550	\$137,051 – \$208,850	\$68,525 – \$104,425	\$117,451 – \$190,200
33%	\$171,551 – \$372,950	\$208,851 – \$372,950	\$104,426 – \$186,475	\$190,201 – \$372,950
35%	\$372,951+	\$372,951+	\$186,476+	\$372,951+

1.2 Java Selections – Boolean Operators, if-else, switch statements

```
import java.util.Scanner;

public class ComputeTax {
    public static void main(String[]
args) {
        // Create a Scanner
        Scanner input = new
Scanner(System.in);
        // Prompt the user to enter filing
status
        System.out.print(
            "(0-single filer, 1-married
jointly,\n" +
            "2-married separately, 3-head of
household)\n" +
            "Enter the filing status: ");
        int status = input.nextInt();
        // Prompt the user to enter
taxable income
        System.out.print("Enter the
taxable income: ");
        double income =
input.nextDouble();
        // Compute tax
        double tax = 0;

        if (status == 0) { // Compute tax for single filers
            if (income <= 8350)
                tax = income * 0.10;
            else if (income <= 33950)
                tax = 8350 * 0.10 + (income - 8350) * 0.15;
            else if (income <= 82250)
                tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
                    (income - 33950) * 0.25;
            else if (income <= 171550)
                tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
                    (82250 - 33950) * 0.25 + (income - 82250) * 0.28;
            else if (income <= 372950)
                tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
                    (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 +
                    (income - 171550) * 0.35;
            else
                tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
                    (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 +
                    (372950 - 171550) * 0.33 + (income - 372950) * 0.35;
        }
        else if (status == 1) { // Compute tax for married file jointly
            // Left as exercise
        }
        else if (status == 2) { // Compute tax for married separately
            // Left as exercise
        }
        else if (status == 3) { // Compute tax for head of household
            // Left as exercise
        }
        else {
            System.out.println("Error: invalid status");
            System.exit(0);
        }

        // Display the result
        System.out.println("Tax is " + (int)(tax * 100) / 100.0);
    }
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

ASSIGNMENT 01 - Problem: Computing Taxes, cont.

```
if (status == 0) {  
    // Compute tax for single filers  
}  
else if (status == 1) {  
    // Compute tax for married file jointly  
}  
else if (status == 2) {  
    // Compute tax for married file separately  
}  
else if (status == 3) {  
    // Compute tax for head of household  
}  
else {  
    // Display wrong status  
}
```

Logical Operators

<i>Operator</i>	<i>Name</i>
!	not
&&	and
	or
^	exclusive or

1.2 Java Selections – Boolean Operators, if-else, switch statements

Truth Table for Operator !

p	!p	Example (assume age = 24, gender = 'M')
true	false	!(age > 18) is false, because (age > 18) is true.
false	true	!(gender != 'F') is true, because (gender != 'F') is false.

p	!p	Example
true	false	!(1 > 2) is true, because (1 > 2) is false.
false	true	!(1 > 0) is false, because (1 > 0) is true.

1.2 Java Selections – Boolean Operators, if-else, switch statements

Truth Table for Operator **&&**

p1	p2	p1 && p2	Example (assume age = 24, gender = 'F')
false	false	false	<u>(age > 18) && (gender == 'F')</u> is true, because <u>(age > 18)</u> and <u>(gender == 'F')</u> are both true.
false	true	false	
true	false	false	<u>(age > 18) && (gender != 'F')</u> is false, because <u>(gender != 'F')</u> is false.
true	true	true	

p1	p2	p1 && p2	Example
false	false	false	(3 > 2) && (5 >= 5) is true, because (3 > 2) and (5 >= 5) are both true.
false	true	false	
true	false	false	(3 > 2) && (5 > 5) is false, because (5 > 5) is false.
true	true	true	

1.2 Java Selections – Boolean Operators, if-else, switch statements

Truth Table for Operator ||

p1	p2	p1 p2	Example (assume age = 24, gender = 'F')
false	false	false	<u>(age > 34) (gender == 'F')</u> is true, because <u>(gender == 'F')</u> is true.
false	true	true	
true	false	true	<u>(age > 34) (gender == 'M')</u> is false, because <u>(age > 34)</u> and <u>(gender == 'M')</u> are both false.
true	true	true	

p1	p2	p1 p2	Example
false	false	false	<u>(2 > 3) (5 > 5)</u> is false, because <u>(2 > 3)</u> and <u>(5 > 5)</u> are both false.
false	true	true	
true	false	true	<u>(3 > 2) (5 > 5)</u> is true, because <u>(3 > 2)</u> is true.
true	true	true	

1.2 Java Selections – Boolean Operators, if-else, switch statements

Truth Table for Operator ^

p1	p2	p1 ^ p2	Example (assume age = 24, gender = 'F')
false	false	false	<u>(age > 34) ^ (gender == 'F')</u> is true, because <u>(age > 34)</u> is false but <u>(gender == 'F')</u> is true.
false	true	true	
true	false	true	<u>(age > 34) (gender == 'M')</u> is false, because <u>(age > 34)</u> and <u>(gender == 'M')</u> are both false.
true	true	false	

1.2 Java Selections – Boolean Operators, if-else, switch statements

Examples

```
System.out.println("Is " + number + " divisible by 2 and 3? " +  
((number % 2 == 0) && (number % 3 == 0)));
```

```
System.out.println("Is " + number + " divisible by 2 or 3? " +  
((number % 2 == 0) || (number % 3 == 0)));
```

```
System.out.println("Is " + number +  
" divisible by 2 or 3, but not both? " +  
((number % 2 == 0) ^ (number % 3 == 0)));
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

The **&** and **|** Operators

If `x` is 1, what is `x` after this expression?

`(x > 1) & (x++ < 10)`

If `x` is 1, what is `x` after this expression?

`(1 > x) && (1 > x++)`

How about `(1 == x) | (10 > x++)`?

`(1 == x) || (10 > x++)`?

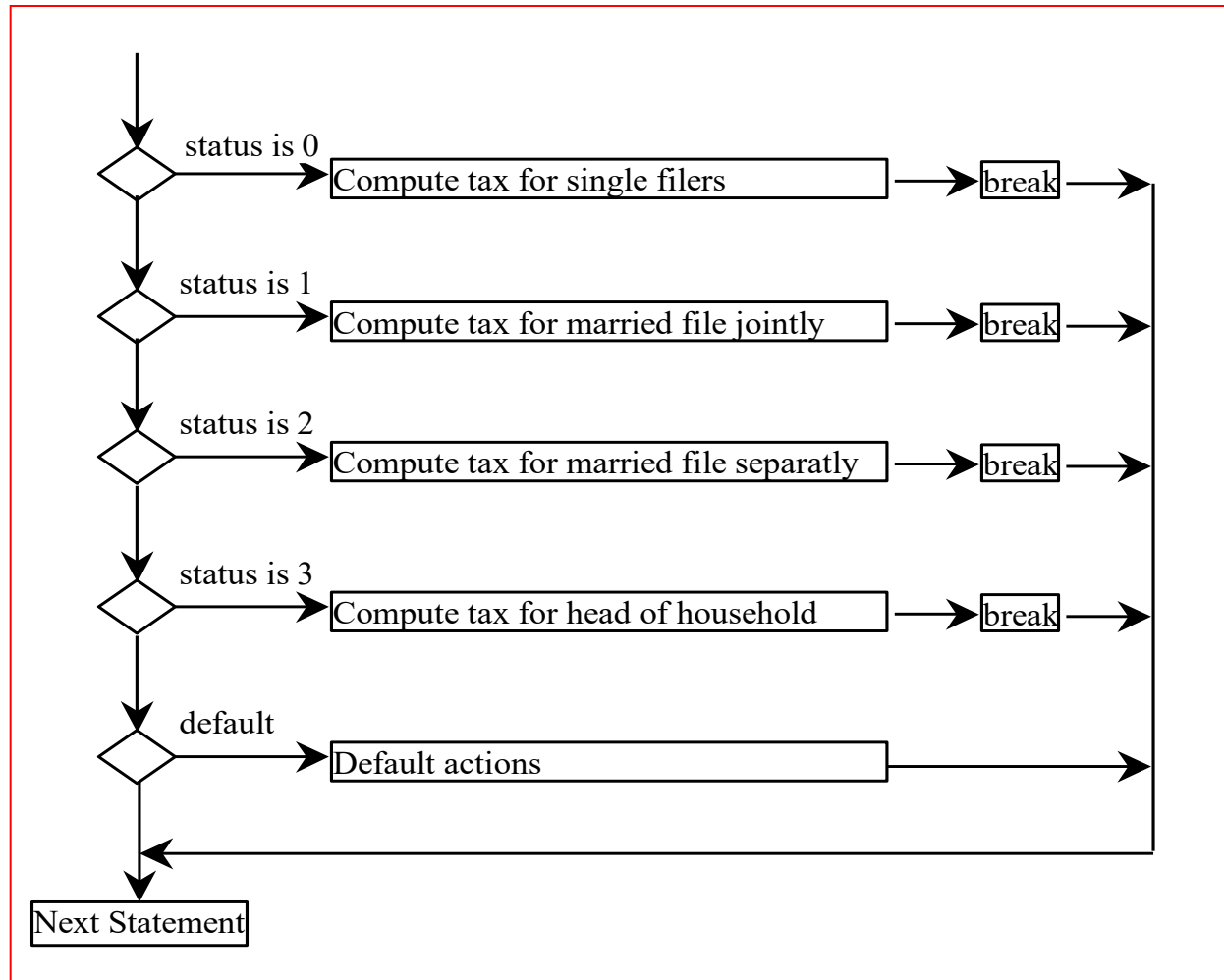
1.2 Java Selections – Boolean Operators, if-else, switch statements

switch Statements

```
switch (status) {  
    case 0: compute taxes for single filers;  
        break;  
    case 1: compute taxes for married file jointly;  
        break;  
    case 2: compute taxes for married file separately;  
        break;  
    case 3: compute taxes for head of household;  
        break;  
    default: System.out.println("Errors: invalid status");  
        System.exit(0);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

switch Statement Flow Chart



1.2 Java Selections – Boolean Operators, if-else, switch statements

switch Statement Rules

The switch-expression must yield a value of char, byte, short, or int type and must always be enclosed in parentheses.

The value1, ..., and valueN must have the same data type as the value of the switch-expression. The resulting statements in the case statement are executed when the value in the case statement matches the value of the switch-expression. Note that value1, ..., and valueN are constant expressions, meaning that they cannot contain variables in the expression, such as $1 + x$.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
    default: statement(s)-for-default;  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

switch Statement Rules

The keyword break is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.

The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
    default: statement(s)-for-default;  
}
```

The case statements are executed in sequential order, but the order of the cases (including the default case) does not matter. However, it is good programming style to follow the logical sequence of the cases and place the default case at the end.

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Suppose ch is 'a':

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

ch is 'a':

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute this line

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute this line

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute this line

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute next statement

```
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'b': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}
```

Next statement;

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Suppose ch is 'a':

```
switch (ch) {  
    case 'a': System.out.println(ch);  
                break;  
    case 'b': System.out.println(ch);  
                break;  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

ch is 'a':

```
switch (ch) {  
    case 'a': System.out.println(ch);  
               break;  
    case 'b': System.out.println(ch);  
               break;  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute this line

```
switch (ch) {  
    case 'a': System.out.println(ch);  
               break;  
    case 'b': System.out.println(ch);  
               break;  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute this line

```
switch (ch) {  
    case 'a': System.out.println(ch);  
        break;  
    case 'b': System.out.println(ch);  
        break;  
    case 'c': System.out.println(ch);  
}
```

1.2 Java Selections – Boolean Operators, if-else, switch statements

Trace switch statement

Execute next statement

```
switch (ch) {  
    case 'a': System.out.println(ch);  
        break;  
    case 'b': System.out.println(ch);  
        break;  
    case 'c': System.out.println(ch);  
}
```

Next statement;

1.2 Java Selections – Boolean Operators, if-else, switch statements

Conditional Operator

```
if (x > 0)
    y = 1;
else
    y = -1;
```

is equivalent to

```
y = (x > 0) ? 1 : -1;
```

(boolean-expression) ? expression1 : expression2

Ternary operator
Binary operator
Unary operator

1.2 Java Selections – Boolean Operators, if-else, switch statements

Conditional Operator

(boolean-expression) ? exp1 : exp2

```
if (num % 2 == 0)
    System.out.println(num + "is even");
else
    System.out.println(num + "is odd");

System.out.println(
    (num % 2 == 0)? num + "is even" :
    num + "is odd");
```

Formatting Output

Use the printf statement.

```
System.out.printf(format, items);
```

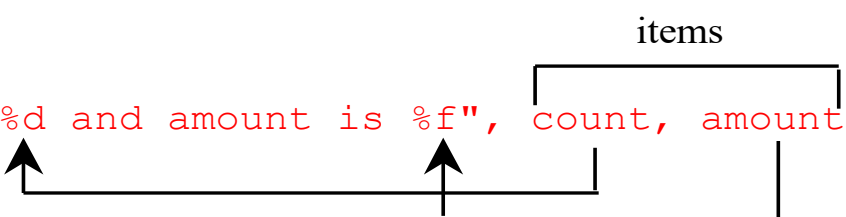
Where format is a string that may consist of substrings and format ***specifiers***. A format ***specifier*** specifies how an item should be displayed. An item may be a numeric value, character, boolean value, or a string. Each ***specifier*** begins with a percent sign.

1.2 Java Selections – Boolean Operators, if-else, switch statements

Frequently-Used Specifiers

Specifier	Output	Example
<u>%b</u>	a boolean value	true or false
<u>%c</u>	a character	'a'
<u>%d</u>	a decimal integer	200
<u>%f</u>	a floating-point number	45.460000
<u>%e</u>	a number in standard scientific notation	4.556000e+01
<u>%s</u>	a string	"Java is cool"
<u>%02x</u>	a hex value	2F

```
int count = 5;  
double amount = 45.56;  
System.out.printf("count is %d and amount is %f", count, amount);
```



display count is 5 and amount is 45.560000

1.2 Java Selections – Boolean Operators, if-else, switch statements

Operator Precedence

- `var++`, `var--`
- `+`, `-` (Unary plus and minus), `++var`, `--var`
- `(type)` Casting
- `!` (Not)
- `*`, `/`, `%` (Multiplication, division, and remainder)
- `+`, `-` (Binary addition and subtraction)
- `<`, `<=`, `>`, `>=` (Comparison)
- `==`, `!=`; (Equality)
- `^` (Exclusive OR)
- `&&` (Conditional AND) Short-circuit AND
- `||` (Conditional OR) Short-circuit OR
- `=`, `+=`, `-=`, `*=`, `/=`, `%=` (Assignment operator)

1.2 Java Selections – Boolean Operators, if-else, switch statements

Operator Precedence and Associativity

The expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

If operators with the same precedence are next to each other, their associativity determines the order of evaluation. All binary operators except assignment operators are left-associative.

Operator Associativity

When two operators with the same precedence are evaluated, the *associativity* of the operators determines the order of evaluation. All binary operators except assignment operators are *left-associative*.

$a - b + c - d$ is equivalent to $((a - b) + c) - d$

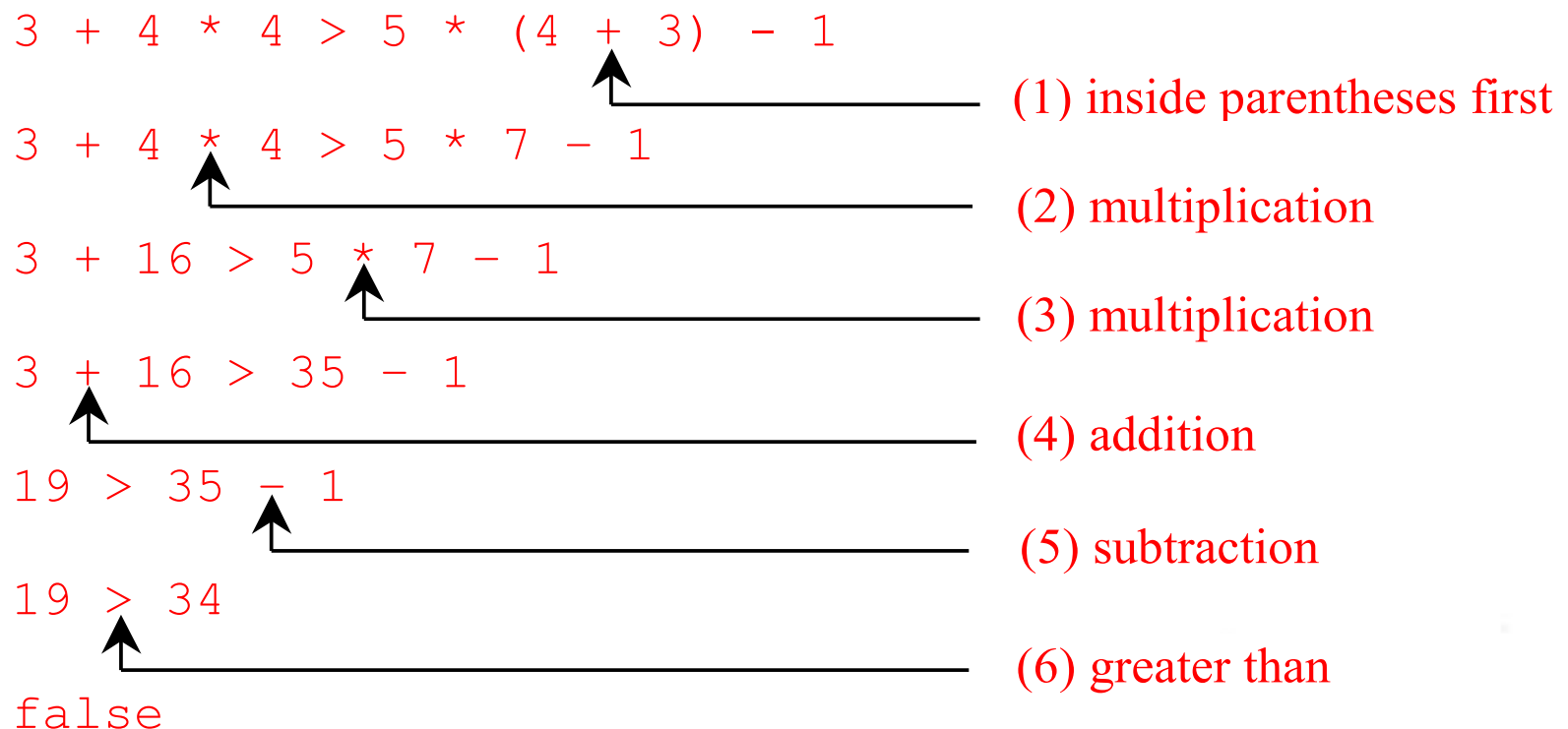
Assignment operators are *right-associative*. Therefore, the expression

$a = b += c = 5$ is equivalent to $a = (b += (c = 5))$

1.2 Java Selections – Boolean Operators, if-else, switch statements

Example

Applying the operator precedence and associativity rule, the expression **3 + 4 * 4 > 5 * (4 + 3) - 1** is evaluated as follows:



1.3 Java Loops – for, while, ... statements

Problem:

100
times

```
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
...  
...  
...  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");  
System.out.println("Welcome to Java!");
```

Solution:

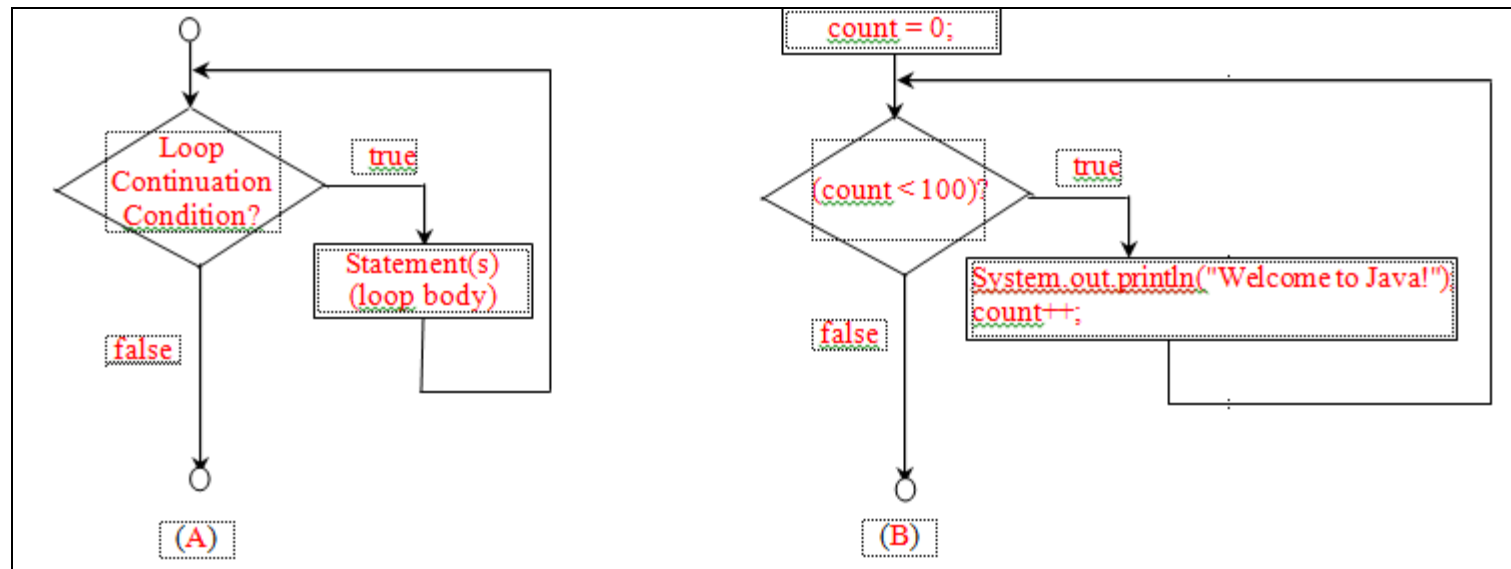
```
int count = 0;  
while (count < 100) {  
    System.out.println("Welcome to Java");  
    count++;  
}
```

1.3 Java Loops – for, while, ... statements

while Loop Flow Chart

```
while (loop-continuation-condition) {  
    // loop-body;  
    Statement(s);  
}
```

```
int count = 0;  
while (count < 100) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```



1.3 Java Loops – for, while, ... statements

Trace while Loop

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```

Initialize count

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
```

(count < 2) is true

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

Print Welcome to Java

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
```

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
```

Increase count by 1
count is 1 now

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

(count < 2) is still true since count is 1

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Print Welcome to Java

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Increase count by 1
count is 2 now

1.3 Java Loops – for, while, ... statements

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

(count < 2) is false since count is 2
now

1.3 Java Loops – for, while, ... statements

Trace while Loop

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

The loop exits. Execute the next statement after the loop.

1.3 Java Loops – for, while, ... statements

Caution

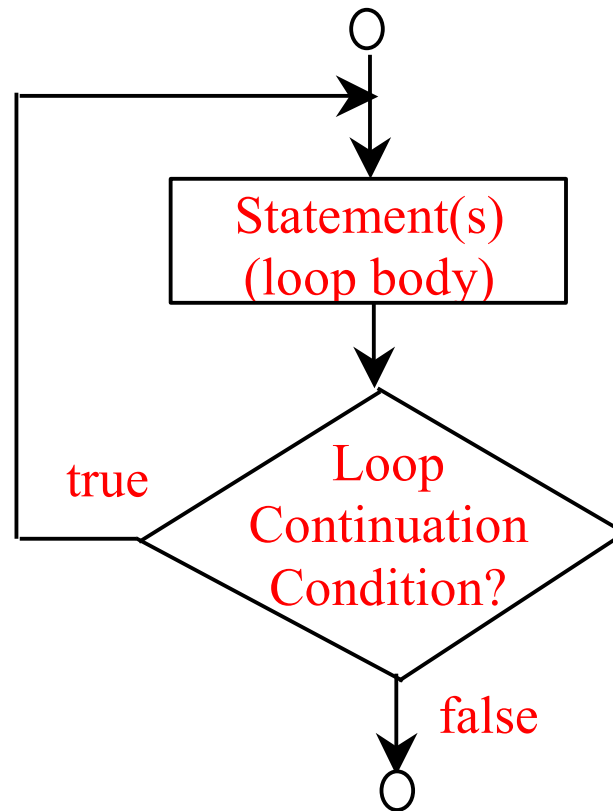
Don't use floating-point values for equality checking in a loop control. Since floating-point values are approximations for some values, using them could result in imprecise counter values and inaccurate results. Consider the following code for computing $1 + 0.9 + 0.8 + \dots + 0.1$:

```
double item = 1; double sum = 0;
while (item != 0) { // No guarantee item will be 0
    sum += item;
    item -= 0.1;
}
System.out.println(sum);
```

Variable `item` starts with 1 and is reduced by 0.1 every time the loop body is executed. The loop should terminate when `item` becomes 0. However, there is no guarantee that `item` will be exactly 0, because the floating-point arithmetic is approximated. This loop seems OK on the surface, but it is actually an infinite loop.

1.3 Java Loops – for, while, ... statements

do-while Loop

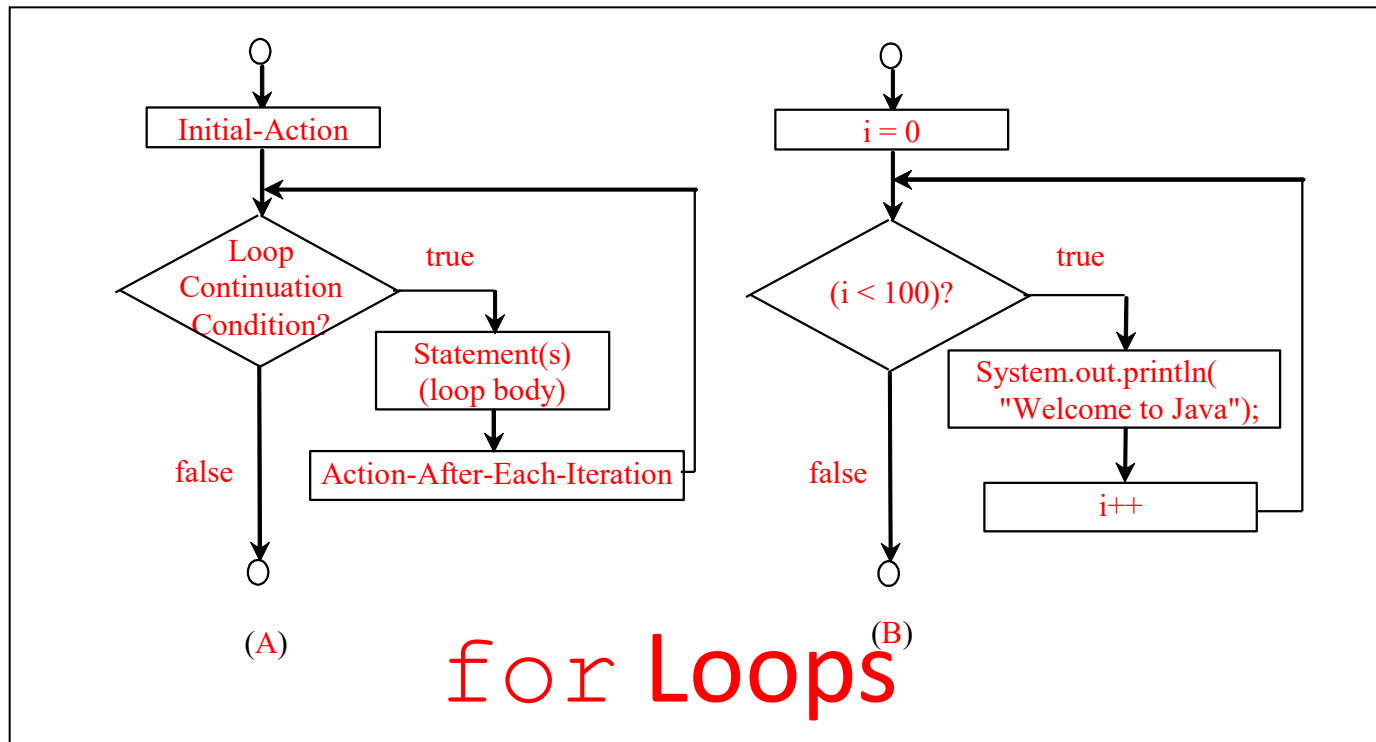


```
do {  
    // Loop body;  
    Statement(s) ;  
} while (loop-continuation-condition) ;
```

1.3 Java Loops – for, while, ... statements

```
for (initial-action; loop-  
    continuation-condition;  
    action-after-each-iteration) {  
    // loop body;  
    Statement(s);  
}
```

```
int i;  
for (i = 0; i < 100; i++) {  
    System.out.println(  
        "Welcome to Java!");  
}
```



1.3 Java Loops – for, while, ... statements

Trace for Loop

```
int i;
```

```
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Declare i

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute initializer
i is now 0

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println( "Welcome to Java!");  
}
```

(i < 2) is true
since i is 0

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Print Welcome to Java

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute adjustment statement
i now is 1

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is still true
since i is 1

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Print Welcome to Java 2nd time

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute adjustment statement
i now is 2

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is false
since i is 2

1.3 Java Loops – for, while, ... statements

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Exit the loop. Execute the next statement after the loop

1.3 Java Loops – for, while, ... statements

Note

The initial-action in a for loop can be a list of zero or more comma-separated expressions. The action-after-each-iteration in a for loop can be a list of zero or more comma-separated statements. Therefore, the following two for loops are correct. They are rarely used in practice, however.

```
for (int i = 1; i < 100; System.out.println(i++));
```

```
for (int i = 0, j = 0; (i + j < 10); i++, j++) {
```

```
    // Do something
```

```
}
```

1.3 Java Loops – for, while, ... statements

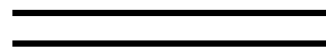
Note

If the loop-continuation-condition in a for loop is omitted, it is implicitly true. Thus the statement given below in (a), which is an infinite loop, is correct. Nevertheless, it is better to use the equivalent loop in (b) to avoid confusion:

```
for ( ; ; ) {  
    // Do something  
}
```

(a)

Equivalent



```
while (true) {  
    // Do something  
}
```

(b)

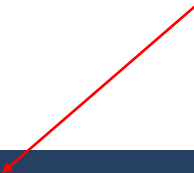
1.3 Java Loops – for, while, ... statements

Caution

Adding a semicolon at the end of the for clause before the loop body is a common mistake, as shown below:

```
for (int i=0; i<10; i++);  
{  
    System.out.println("i is " + i);  
}
```

Logic Error



1.3 Java Loops – for, while, ... statements

Similarly, the following loop is also wrong:

```
int i=0;  
while (i < 10); ← Logic Error  
{  
    System.out.println("i is " + i);  
    i++;  
}
```

Caution, cont.

In the case of the do loop, the following semicolon is needed to end the loop.

```
int i=0;  
do {  
    System.out.println("i is " + i);  
    i++;  
} while (i<10); ← Correct
```

1.3 Java Loops – for, while, ... statements

Which Loop to Use?

The three forms of loop statements, while, do-while, and for, are expressively equivalent; that is, you can write a loop in any of these three forms. For example, a while loop in (a) in the following figure can always be converted into the following for loop in (b):

```
while (loop-continuation-condition) {  
    // Loop body  
}
```

(a)

Equivalent

```
for ( ; loop-continuation-condition; )  
    // Loop body  
}
```

(b)

A for loop in (a) in the following figure can generally be converted into the following while loop in (b) except in certain special:

```
for (initial-action;  
     loop-continuation-condition;  
     action-after-each-iteration) {  
    // Loop body;  
}
```

(a)

Equivalent

```
initial-action;  
while (loop-continuation-condition) {  
    // Loop body;  
    action-after-each-iteration;  
}
```

(b)

1.3 Java Loops – for, while, ... statements

Recommendations

Use the one that is most intuitive and comfortable for you. In general, a for loop may be used if the number of repetitions is known, as, for example, when you need to print a message 100 times. A while loop may be used if the number of repetitions is not known, as in the case of reading the numbers until the input is 0. A do-while loop can be used to replace a while loop if the loop body has to be executed before testing the continuation condition.

ASSIGNMENT 02 - Problem: Write a program that uses nested for loops to print a multiplication table – nested loops.

1.4 Java Methods – defining, signature, formal / actual parameters

Problem

```
int sum = 0;
for (int i = 1; i <= 10; i++)
    sum += i;
System.out.println("Sum from 1 to 10 is " + sum);

sum = 0;
for (int i = 20; i <= 30; i++)
    sum += i;
System.out.println("Sum from 20 to 30 is " + sum);

sum = 0;
for (int i = 35; i <= 45; i++)
    sum += i;
System.out.println("Sum from 35 to 45 is " + sum);
```

1.4 Java Methods – defining, signature, formal / actual parameters

Problem

```
int sum = 0;  
for (int i = 1; i <= 10; i++)  
    sum += i;
```

```
System.out.println("Sum from 1 to 10 is " + sum);
```

```
sum = 0;  
for (int i = 20; i <= 30; i++)  
    sum += i;
```

```
System.out.println("Sum from 20 to 30 is " + sum);
```

```
sum = 0;  
for (int i = 35; i <= 45; i++)  
    sum += i;
```

```
System.out.println("Sum from 35 to 45 is " + sum);
```

1.4 Java Methods – defining, signature, formal / actual parameters

Solution

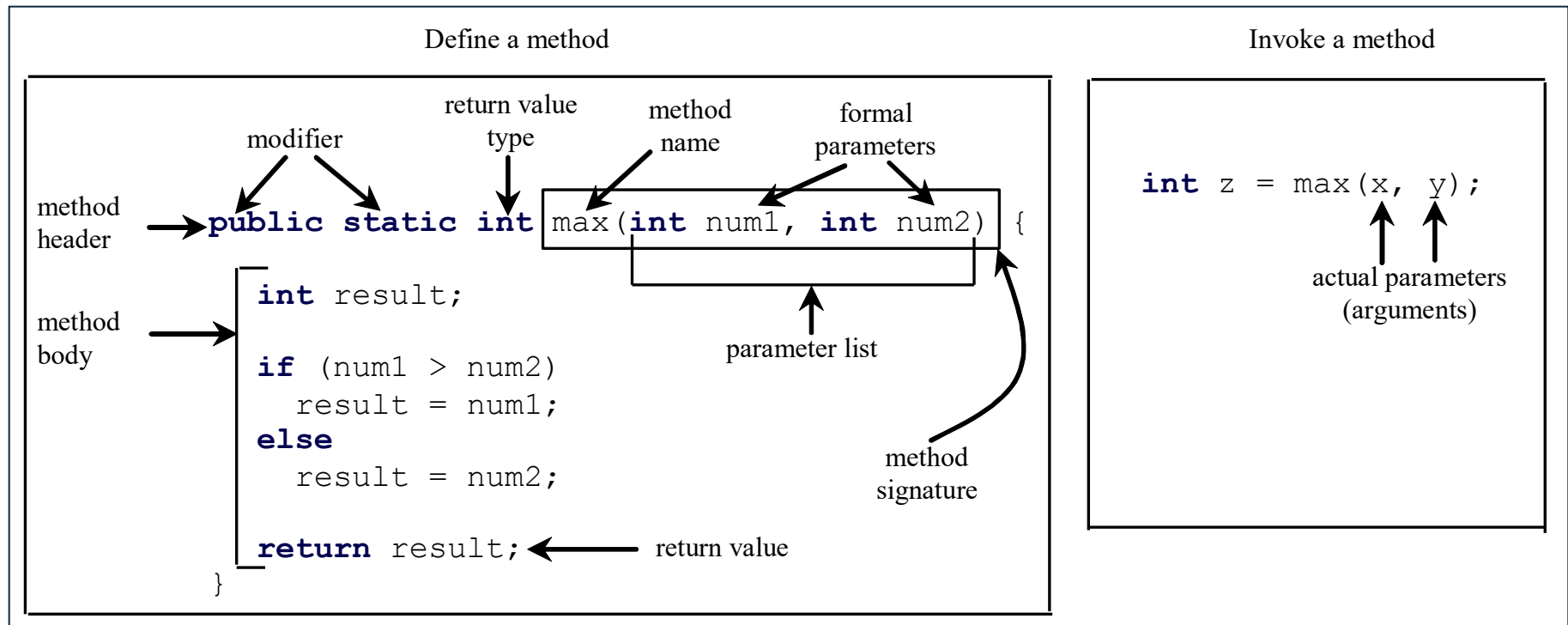
```
public static int sum(int i1, int i2) {  
    int sum = 0;  
    for (int i = i1; i <= i2; i++)  
        sum += i;  
    return sum;  
}
```

```
public static void main(String[] args) {  
    System.out.println("Sum from 1 to 10 is " + sum(1, 10));  
    System.out.println("Sum from 20 to 30 is " + sum(20, 30));  
    System.out.println("Sum from 35 to 45 is " + sum(35, 45));  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

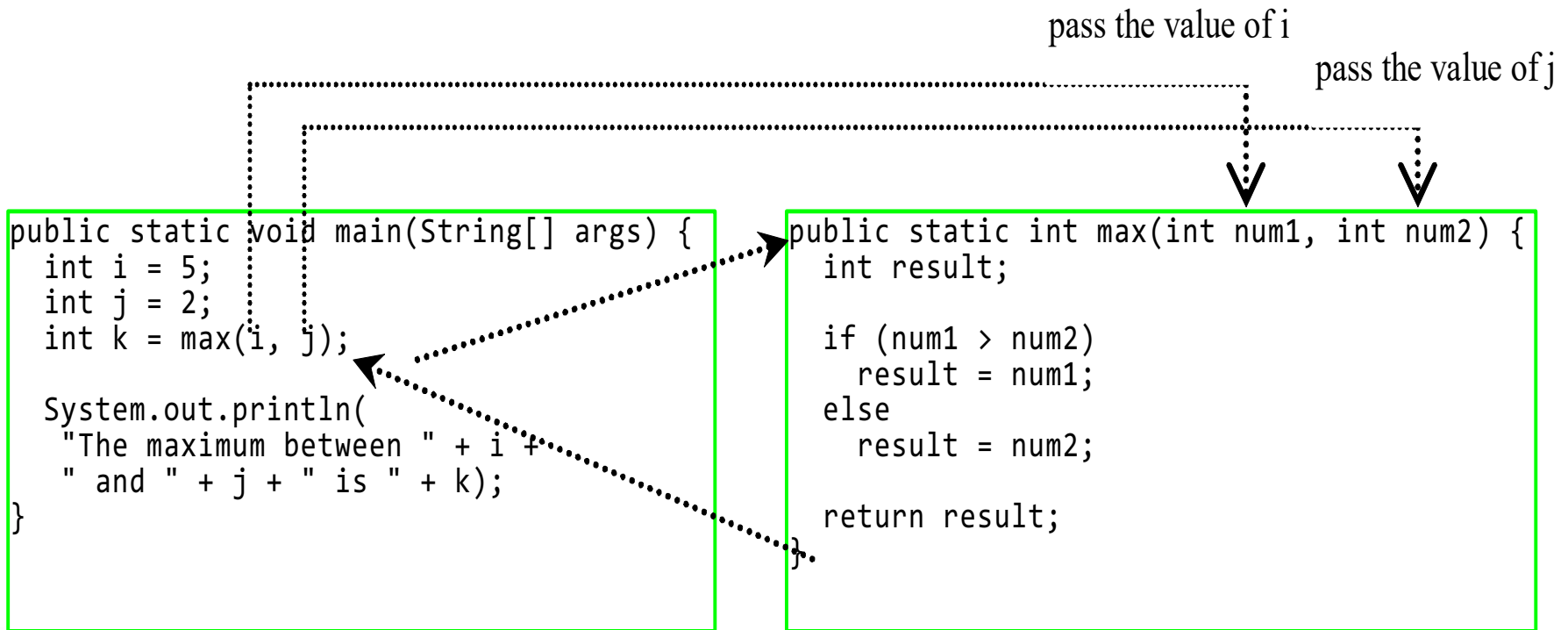
Defining Methods

A method is a collection of statements that are grouped together to perform an operation.



1.4 Java Methods – defining, signature, formal / actual parameters

Calling Methods



1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

i is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

j is now 2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

invoke max(i, j)
Pass the value of i to num1
Pass the value of j to num2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

declare variable result

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

(num1 > num2) is true since num1 is 5 and num2 is 2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

result is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

return result, which is 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

return max(i, j) and assign the
return value to k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Method Invocation

Execute the print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

CAUTION

A return statement is required for a value-returning method. The method shown below in (a) is logically correct, but it has a compilation error because the Java compiler thinks it possible that this method does not return any value.

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```

(a)

Should be

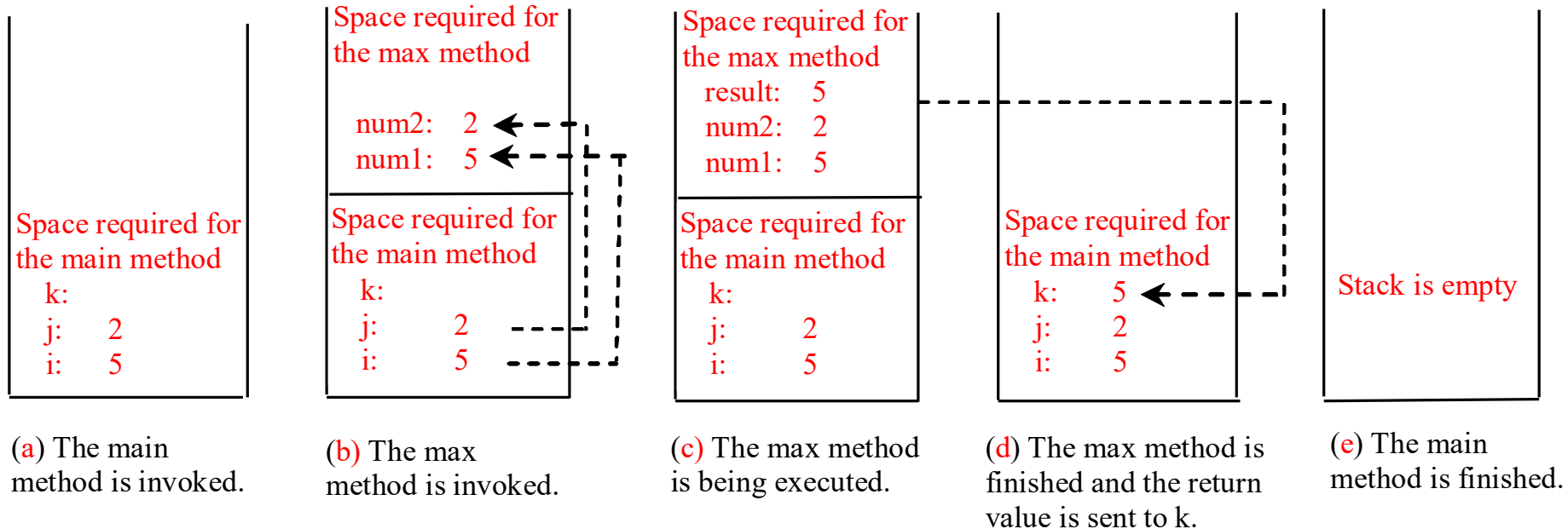
```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else  
        return -1;  
}
```

(b)

To fix this problem, delete if (n < 0) in (a), so that the compiler will see a return statement to be reached regardless of how the if statement is evaluated.

1.4 Java Methods – defining, signature, formal / actual parameters

Call Stacks



1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

i is declared and initialized

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

i: 5

The main method
is invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

j is declared and initialized

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

j: 2
i: 5

The main method
is invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

Declare k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

Invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

pass the values of i and j to num1
and num2

num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

pass the values of i and j to num1
and num2

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

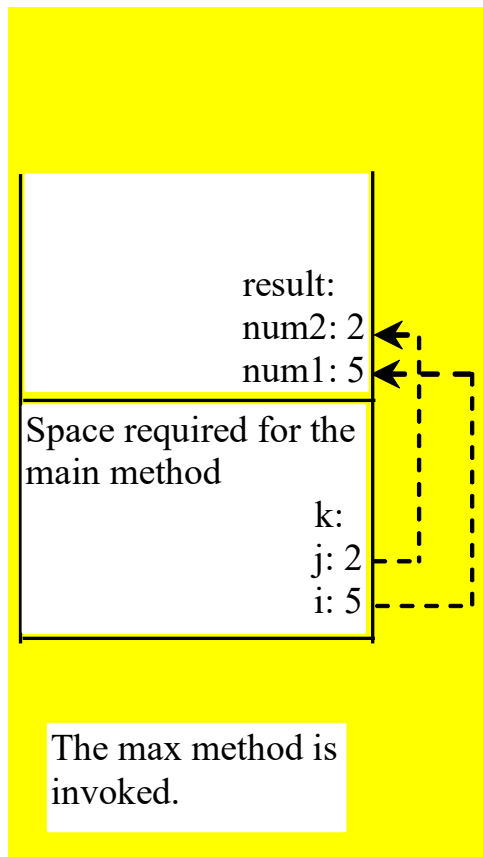
1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

(num1 > num2) is true

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```



1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

Assign num1 to result

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2)  
int result;  
  
if (num1 > num2)  
    result = num1;  
else  
    result = num2;  
  
return result;  
}
```

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

Return result and assign it to k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k: 5
j: 2
i: 5

The max method is
invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Trace Call Stack

Execute print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:5
j: 2
i: 5

The main method
is invoked.

1.4 Java Methods – defining, signature, formal / actual parameters

Passing Parameters

```
public static void nPrintln(String message, int n) {  
    for (int i = 0; i < n; i++)  
        System.out.println(message);  
}
```

Suppose you invoke the method using

`nPrintln("Welcome to Java", 5);`

What is the output?

Suppose you invoke the method using

`nPrintln("Computer Science", 15);`

What is the output?

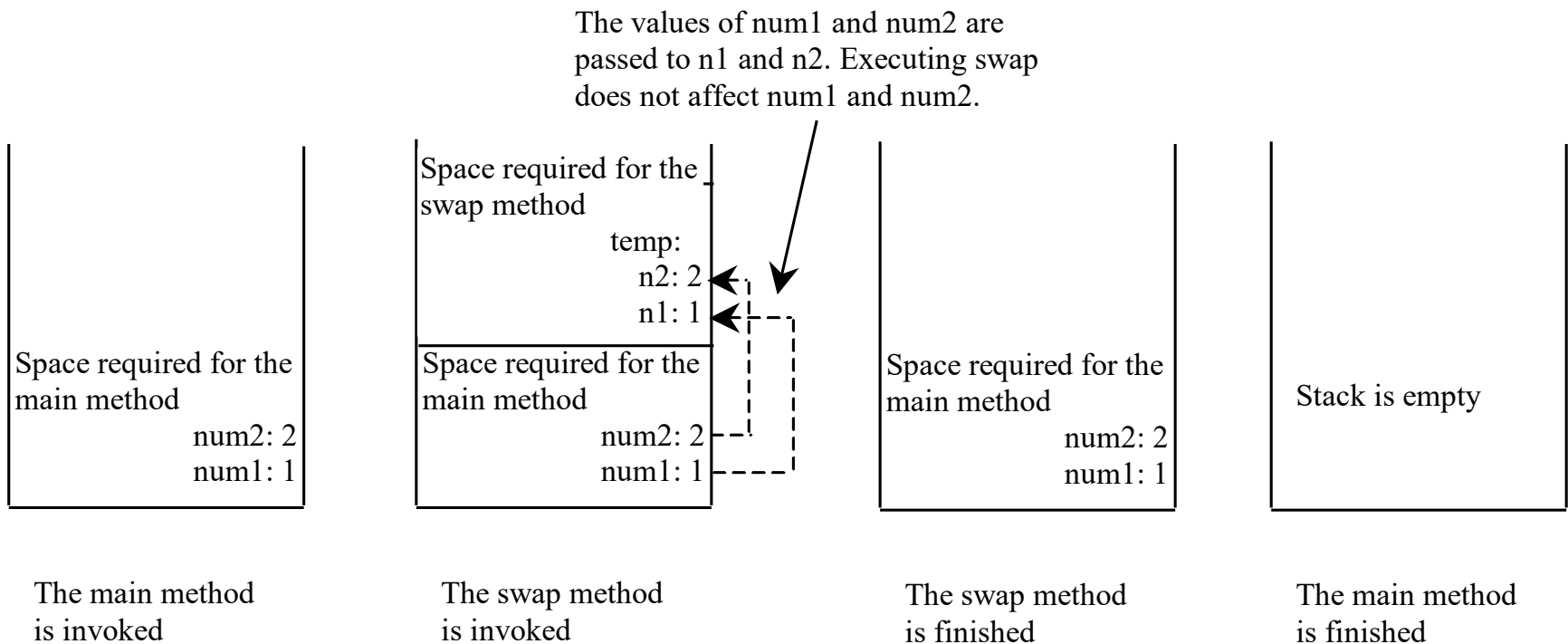
1.4 Java Methods – defining, signature, formal / actual parameters

Passing parameters by value

```
public class TestPassByValue {  
    /** Main method */  
    public static void main(String[] args) {  
        // Declare and initialize variables  
        int num1 = 1;  
        int num2 = 2;  
  
        System.out.println("Before invoking the swap method, num1 is " +  
            num1 + " and num2 is " + num2);  
  
        // Invoke the swap method to attempt to swap two variables  
        swap(num1, num2);  
  
        System.out.println("After invoking the swap method, num1 is " +  
            num1 + " and num2 is " + num2);  
    }  
  
    /** Swap two variables */  
    public static void swap(int n1, int n2) {  
        System.out.println("\t\tInside the swap method");  
        System.out.println("\t\tBefore swapping n1 is " + n1  
            + " n2 is " + n2);  
  
        // Swap n1 with n2  
        int temp = n1;  
        n1 = n2;  
        n2 = temp;  
  
        System.out.println("\t\tAfter swapping n1 is " + n1  
            + " n2 is " + n2);  
    }  
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Call Stacks – Passing parameters by value



1.4 Java Methods – defining, signature, formal / actual parameters

Overloading methods - Ambiguous Invocation

```
public class AmbiguousOverloading {
    public static void main(String[] args) {
        System.out.println(max(1, 2));
    }

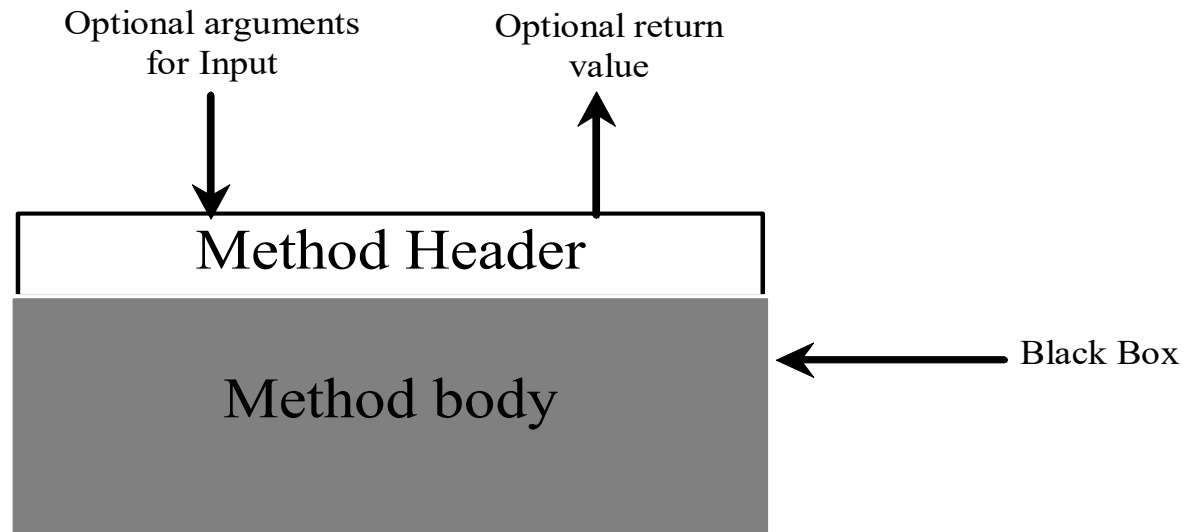
    public static double max(int num1, double num2) {
        if (num1 > num2)
            return num1;
        else
            return num2;
    }

    public static double max(double num1, int num2) {
        if (num1 > num2)
            return num1;
        else
            return num2;
    }
}
```

1.4 Java Methods – defining, signature, formal / actual parameters

Method Abstraction

A programmer can think of the method body as a black box that contains the detailed implementation for the method.



Benefits of using methods:

- Write a method once and reuse it anywhere.
- Information hiding. Hide the implementation from the user.
- Reduce complexity.

The Math Class

Case study:

- Class constants:
 - `PI`
 - `E`
- Class methods:
 - Trigonometric Methods
 - Exponent Methods
 - Rounding Methods
 - `min`, `max`, `abs`, and random Methods

Trigonometric Methods

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

Examples:

`Math.sin(0)` returns 0.0

`Math.sin(Math.PI / 6)` returns 0.5

`Math.sin(Math.PI / 2)` returns 1.0

`Math.cos(0)` returns 1.0

`Math.cos(Math.PI / 6)` returns
0.866

`Math.cos(Math.PI / 2)` returns 0

1.4 Java Methods – defining, signature, formal / actual parameters

Exponent Methods

- `exp(double a)`
Returns e raised to the power of a .
- `log(double a)`
Returns the natural logarithm of a .
- `log10(double a)`
Returns the 10-based logarithm of a .
- `pow(double a, double b)`
Returns a raised to the power of b .
- `sqrt(double a)`
Returns the square root of a .

Examples:

```
Math.exp(1) returns 2.71
Math.log(2.71) returns 1.0
Math.pow(2, 3) returns 8.0
Math.pow(3, 2) returns 9.0
Math.pow(3.5, 2.5) returns
    22.91765
Math.sqrt(4) returns 2.0
Math.sqrt(10.5) returns 3.24
```

1.4 Java Methods – defining, signature, formal / actual parameters

min, max, and abs

- `max(a, b)` and `min(a, b)`
Returns the maximum or minimum of two parameters.
- `abs(a)`
Returns the absolute value of the parameter.
- `random()`
Returns a random double value in the range [0.0, 1.0).

Examples:

`Math.max(2, 3)` returns 3

`Math.max(2.5, 3)` returns 3.0

`Math.min(2.5, 3.6)` returns 2.5

`Math.abs(-2)` returns 2

`Math.abs(-2.1)` returns 2.1

Section Conclusion

Fact: **Java is natural**

In few **samples** it is simple to remember:
selections, loops, methods – it is like in
C/C++ syntax.





Java OOP

Java OOP & JSE

2. Summary of Java SE - OOP

Analogy with C++ and it will be reloaded in next lecture (C02)

What is a class?

What is a package of classes in Java?

What is an object / instance?

How many bytes has an object inside JVM?

Why do we need clone, equals and hash methods?

Demo & memory model for the ***Certificate*** Java class

2. Summary of Java SE - OOP

Polymorphism – “the ability to have multiple forms” is obtaining through:

- The overriding (not overloading; overloading is a form of polymorphism) of the methods from a Java class
- “Pure” form of polymorphism has to have the following conditions:
 - Inheritance mechanism – “**extends** is the key word”
 - Virtual Methods – “in Java by default”
 - Overriding the virtual methods
 - Using the pointer / references to the objects – “in Java by default”

Interface – “contract between objects of the class that implements the interface and another objects of another classes that interact with objects from the implementation class” – has the following features:

- static fields
- Static and non-static methods prototypes – declaration but NOT implementation
- The implementation class use the “**implements**” keyword
- It is possible to have interface as type – declare objects with type interface but you must call the constructor methods from a real Java class

2. Summary of Java SE - OOP

Abstract Class – “a class that have at least one method abstract” – has the following features:

- At least one abstract method – keyword “**abstract**”
- May contain standard static and non-static fully implemented methods
- You may declare objects from an abstract class but you can NOT instantiate them, you should use constructor method from a real Java class

Pay attention @: Objects vs. vectors / arrays of objects + null pointer exception

DEMO

Compile in command prompt & Eclipse – Shallow Copy vs. Deep Copy Example.

Section Conclusions

Object Oriented Programming in Java – class, object, object's deep vs. shallow copy, interface, interface as type, abstract class, inheritance, polymorphism.

Class is used to encapsulate attributes (fields | features | characteristics) and methods (behavior | 'interface') in order to produce instances / objects.

Object is a instance of a class with certain values for attributes and with same methods class for all the generated instances.

In OOP there are two major relationships: “has a” (composition) and “is a” (inheritance)



Share knowledge, Empowering Minds

Multi-paradigm Programming Intro



3.1 Multi-paradigm programming

Some paradigms are concerned mainly with implications for the [execution model](#) of the language, such as allowing [side effects](#), or whether the sequence of operations is defined by the execution model. Other paradigms are concerned mainly with the way that code is organized, such as grouping a code into units along with the state that is modified by the code. Yet others are concerned mainly with the style of syntax and grammar.

Common programming paradigms include:

[imperative](#) which allows side effects,

[functional](#) which disallows side effects,

[declarative](#) which does not state the order in which operations execute,

[object-oriented](#) which groups code together with the state the code modifies,

[procedural](#) which groups code into functions,

[logic](#) which has a particular style of execution model coupled to a particular style of syntax and grammar, and

[symbolic](#) programming which has a particular style of syntax and grammar.^{[1][2][3]}

For example, languages that fall into the **imperative paradigm** have two main features: they state the order in which operations occur, with constructs that explicitly control that order, and they allow side effects, in which state can be modified at one point in time, within one unit of code, and then later read at a different point in time inside a different unit of code. The communication between the units of code is not explicit. Meanwhile, in **object-oriented** programming, code is organized into [objects](#) that contain state that is only modified by the code that is part of the object. Most object-oriented languages are also imperative languages. In contrast, languages that fit the **declarative paradigm** do not state the order in which to execute operations. Instead, they supply a number of operations that are available in the system, along with the conditions under which each is allowed to execute.

3.1 Multi-paradigm programming

Just as [software engineering](#) (as a process) is defined by differing *methodologies*, so the [programming languages](#) (as models of computation) are defined by differing *paradigms*. Some languages are designed to support one paradigm ([Smalltalk](#) supports object-oriented programming, [Haskell](#) supports functional programming), while other programming languages support multiple paradigms (such as [Object Pascal](#), [C++](#), [Java](#), [C#](#), [Scala](#), [Visual Basic](#), [Common Lisp](#), [Scheme](#), [Perl](#), [PHP](#), [Python](#), [Ruby](#), [Oz](#), and [F#](#)). For example, programs written in C++, Object Pascal or PHP can be purely [procedural](#), purely [object-oriented](#), or can contain elements of both or other paradigms. Software designers and programmers decide how to use those paradigm elements.

In object-oriented programming, programs are treated as a set of interacting objects. In [functional programming](#), programs are treated as a sequence of stateless function evaluations. When programming computers or systems with many processors, in [process-oriented programming](#), programs are treated as sets of concurrent processes acting on logically shared [data structures](#).

Many programming paradigms are as well known for the techniques they *forbid* as for those they *enable*. For instance, pure functional programming disallows use of [side-effects](#), while [structured programming](#) disallows use of the [goto](#) statement. Partly for this reason, new paradigms are often regarded as doctrinaire or overly rigid by those accustomed to earlier styles.^[5] Yet, avoiding certain techniques can make it easier to understand program behavior, and to [prove theorems](#) about program correctness.

Programming paradigms can also be compared with [programming models](#) which allow invoking an [execution model](#) by using only an API. Programming models can also be classified into paradigms, based on features of the execution model. For [parallel computing](#), using a programming model instead of a language is common. The reason is that details of the parallel hardware leak into the abstractions used to program the hardware. This causes the programmer to have to map patterns in the algorithm onto patterns in the execution model (which have been inserted due to leakage of hardware into the abstraction). As a consequence, no one parallel programming language maps well to all computation problems. It is thus more convenient to use a base sequential language and insert API calls to parallel execution models, via a programming model. Such parallel programming models can be classified according to abstractions that reflect the hardware, such as shared memory, distributed memory with message passing, notions of *place* visible in the code, and so forth. These can be considered flavors of programming paradigm that apply to only parallel languages and programming models.

3.1 Multi-paradigm programming

A concise reference for the programming paradigms listed here – copyright Wiki:

- [Concurrent programming](#) – have language constructs for concurrency, these may involve multi-threading, support for distributed computing, message passing, shared resources (including shared memory), or futures
 - [Actor programming](#) – concurrent computation with *actors* that make local decisions in response to the environment (capable of selfish or competitive behavior)
- [Constraint programming](#) – relations between variables are expressed as constraints (or constraint networks), directing allowable solutions (uses constraint satisfaction or [simplex algorithm](#))
- [Dataflow programming](#) – forced recalculation of formulas when data values change (e.g. [spreadsheets](#))
- [Declarative programming](#) – describes actions (e.g. [HTML](#) describes a page but not how to actually display it)
- [Distributed programming](#) – have support for multiple autonomous computers that communicate via computer networks
- [Functional programming](#) – uses evaluation of mathematical functions and avoids state and mutable data
- [Generic programming](#) – uses algorithms written in terms of to-be-specified-later types that are then instantiated as needed for specific types provided as parameters
- [Imperative programming](#) – explicit statements that change a program state
- [Logic programming](#) – uses explicit mathematical logic for programming
- [Metaprogramming](#) – writing programs that write or manipulate other programs (or themselves) as their data, or that do part of the work at compile time that would otherwise be done at runtime
 - [Template metaprogramming](#) – metaprogramming methods in which templates are used by a compiler to generate temporary source code, which is merged by the compiler with the rest of the source code and then compiled
 - [Reflective programming](#) – metaprogramming methods in which a program modifies or extends itself

3.1 Multi-paradigm programming

- Object-oriented programming – uses data structures consisting of data fields and methods together with their interactions (objects) to design programs
 - Class-based – object-oriented programming in which inheritance is achieved by defining classes of objects, versus the objects themselves
 - Prototype-based – object-oriented programming that avoids classes and implements inheritance via cloning of instances
- Pipeline programming – a simple syntax change to add syntax to nest function calls to language originally designed with none
- Rule-based programming – a network of rules of thumb that comprise a knowledge base and can be used for expert systems and problem deduction & resolution
- Visual programming – manipulating program elements graphically rather than by specifying them textually (e.g. Simulink or Scratch from MIT); also termed *diagrammatic programming*

3.1 Multi-paradigm programming

List of multi-paradigm programming languages

Language	Number of Paradigms	Concurrent	Constraints	Dataflow	Declarative	Distributed	Functional	Meta-programming	Generic	Imperative	Logic	Reflection	Object-oriented	Pipelines	Visual	Rule-based	Other paradigms
C++	7 (15)	Yes ^{[8][9][10]}	Library ^[11]	Library ^{[12][13]}	Library ^{[14][15]}	Library ^{[16][17]}	Yes	Yes ^[18]	Yes ^[a 3]	Yes	Library ^{[19][20]}	Library ^[21]	Yes ^[a 2]	Yes ^[22]	No	Library ^[23]	Array (multi-dimensional; using STL)
C#	6 (7)	Yes	No	Library ^[a 4]	No	No	Yes ^[a 5]	No	Yes	Yes	No	Yes	Yes ^[a 2]	No	No	No	reactive ^[a 6]
ECMAScript ^{[27][28]} (ActionScript, E4X, JavaScript, JScript)	4 (5)	partial (promises, native extensions) ^[a 8]	No	No	No	No	Yes	No	No	Yes	No	Yes	Yes ^[a 9]	No	No	No	reactive ^[a 10]
Fortran	4 (5)	Yes	No	No	No	No	Yes ^[a 11]	No	Yes ^[a 12]	No	No	No	Yes ^[a 2]	No	No	No	Array (multi-dimensional)
Go	4	Yes	No	No	No	No	No	No	No	Yes	No	Yes	No	Yes	No	No	No
Haskell	2?	Yes	Library ^[29]	No	Yes	Library ^[30]	Yes (lazy)	No	Yes	Yes	No	No	No	No	Yes	No	Template:Reactive, dependent types (partial)
Java	6	Yes	Library ^[31]	Library ^[32]	No	No	Yes	No	Yes	Yes	No	Yes	Yes ^[a 2]	No	No	No	No
Kotlin	8	Yes	No	No	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes	Yes	No	No	No
Python	5 (10)	Library ^{[65][66]}	Library ^[67]	No	No	Library ^[68]	Partial	Yes ^{[69][70]}	Yes ^{[71][72]}	Yes	Library ^[73]	Yes	Yes ^[a 2]	No	No	No	structured
R	4	No	No	No	No	No	Yes	No	No	Yes	No	Yes	Yes	Yes ^[74]	No	No	Array (multi-dimensional)
Lua ^[citation needed]	3	No	No	No	No	No	Yes	No	No	Yes	No	No	Yes ^[a 9]	No	No	No	No
MATLAB	6 (10)	Toolbox ^[41]	Toolbox ^[42]	Yes ^[43]	No	Toolbox ^[44]	No	Yes ^[45]	Yes ^[46]	No	No	Yes ^[47]	Yes ^[48]	No	Yes ^[49]	No	Array (multi-dimensional)
Swift	3	No	No	No	No	No	Yes	No	No	Yes	No	No	Yes ^[a 2]	No	No	No	block-structured
Visual Basic .NET	6 (7)	Yes	No	Library ^[a 4]	No	No	Yes	No	Yes	Yes	No	Yes	Yes ^[a 2]	No	No	No	reactive ^[a 6]
Windows PowerShell	6	No	No	No	No	No	Yes	No	Yes	Yes	No	Yes	Yes ^[a 2]	Yes	No	No	No
Wolfram Language & Mathematica	13 ^[82] (14)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	No	Yes	Knowledge Based

3.1 Multi-paradigm programming

Markup languages
(only Datastructures)

More declarative
Paradigms

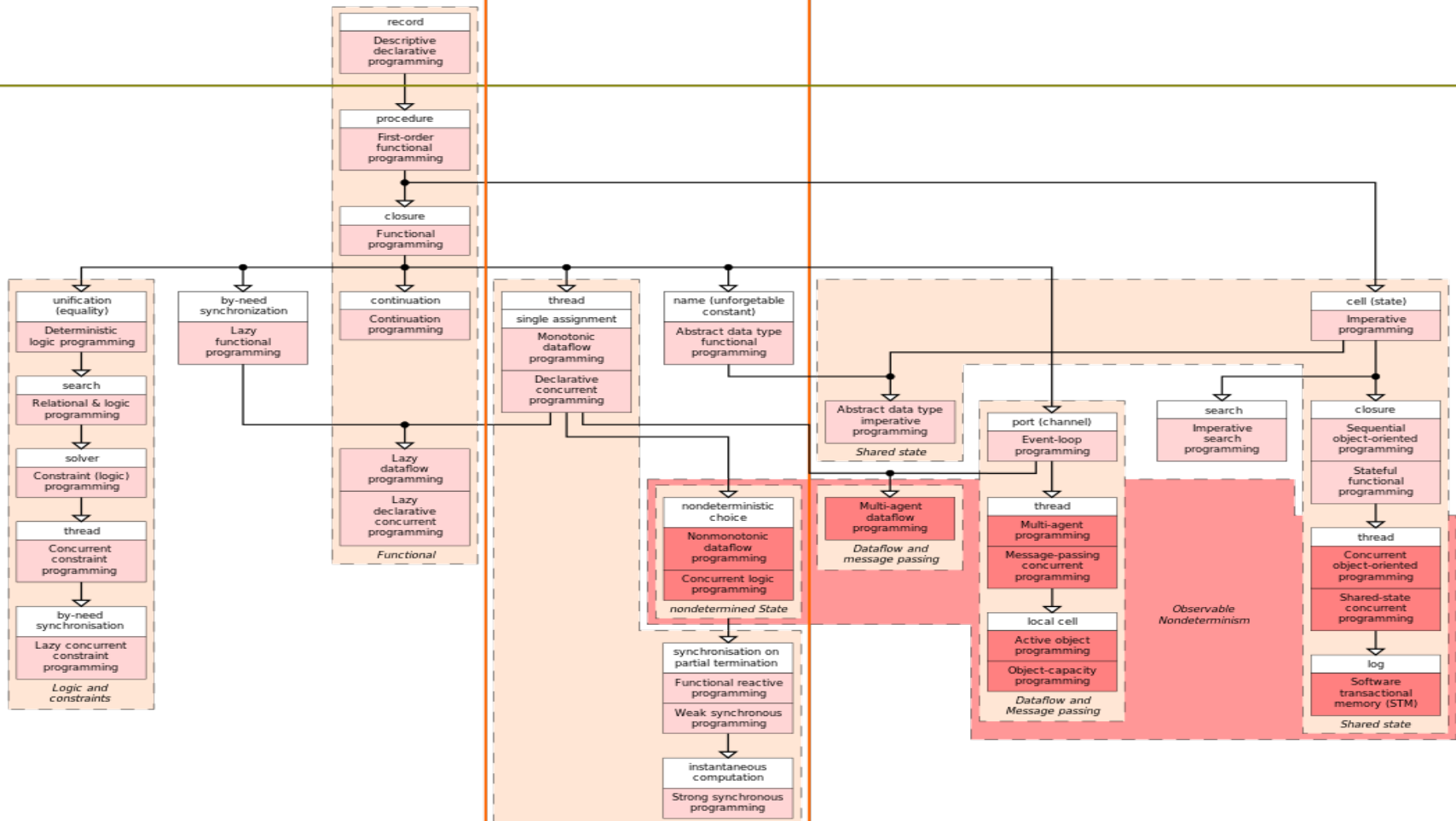
Unnamed state
(sequential or concurrent)

Undeterministic
state

Named
state

More Imperative
Paradigms

Turing complete
Languages



3.2 JVM Languages

This **list of JVM Languages** comprises notable computer [programming languages](#) that are used to produce computer [software](#) that runs on the [Java virtual machine](#) (JVM). Some of these languages are [interpreted](#) by a [Java](#) program, and some are compiled to [Java bytecode](#) and [JIT-compiled](#) during execution as regular Java programs to improve performance.

The JVM was initially designed to support only the programming language Java. However, as time passed, ever more languages were adapted or designed to run on the [Java platform](#).

JVM based High-profile programming languages

Apart from the [Java language](#), the most common or well-known other JVM languages are:

- [Clojure](#), a [functional Lisp](#) dialect
- ***** [Apache Groovy](#)**, a dynamic programming and [scripting language](#) (**Jenkins**)
- [JRuby](#), an implementation of [Ruby](#)
- [Jython](#), an implementation of [Python](#)
- ***** [Kotlin](#)**, a statically-typed language from [JetBrains](#), the developers of [IntelliJ IDEA](#) (**Android Mobile Apps**)
- ***** [Scala](#)**, a [statically-typed](#) [object-oriented](#) and [functional programming](#) language (**Cloud – IaaS / PaaS / CaaS over IaaS, HA/HPC scalable computing, Big Data, DataMining, Collective Intelligence....**)

3.2 JVM Languages

JVM implementations of existing languages

Language	Java implementations
<u>Ada</u>	<u>JGNAT</u>
<u>Arden Syntax</u>	<u>Arden2ByteCode</u>
<u>COBOL</u>	<u>Micro Focus Visual COBOL</u>
<u>ColdFusion Markup Language</u> (CF ML)	<u>Adobe ColdFusion</u> <u>Railo</u> <u>Lucee</u> Open <u>BlueDragon</u>
<u>Common Lisp</u>	Armed Bear Common Lisp
<u>Cypher</u>	<u>Neo4j</u>
<u>JavaScript</u>	<u>Rhino</u> <u>Nashorn</u>
<u>Mercury</u>	<u>Mercury</u> (Java grade)
<u>Oberon</u>	<u>Component Pascal</u>
<u>Pascal</u>	<u>MIDletPascal</u> <u>Oxygene</u>
<u>Perl 6</u>	<u>Rakudo Perl 6</u>
<u>PHP</u>	<u>Quercus</u>

Language	Java implementations
<u>Prolog</u>	<u>JIProlog</u> <u>TuProlog</u>
<u>Python</u>	<u>Jython</u>
<u>R</u>	<u>Renjin</u>
<u>Rexx</u>	<u>NetRexx</u>
<u>Ruby</u>	<u> JRuby</u> <u>Mirah</u>
<u>Scheme</u>	<u>Bigloo</u> <u>Kawa</u> <u>SISC</u> <u>JScheme</u>
<u>Tcl</u>	<u>Jacl</u>
<u>Visual Basic</u>	<u>Jabaco</u>

3.3 Repositories: Git from Linux VM

Mandatory steps to get the lectures and the labs: (Optional full Git presentation

@ISM – Cyber-Security Master – www.ism.ase.ro by Cristian Toma & Casian Andrei @ Oracle):

1. Download & install the Linux Ubuntu 16.04 LTS VM OVA appliance from Google drive:

2. In Ubuntu Linux VM download the Git repo with the sources for the lecture & labs:

<https://github.com/critoma/javase> | <http://acs.ase.ro/java>

Optional – set your e-mail and Git global username if you want to contribute:

git config --global user.email cristian.toma@ie.ase.ro

git config --global user.name "critoma"

Mandatory – clone or pull the latest Git repo updates OR Download-ZIP (web):

git clone <https://github.com/critoma/javase.git>

git pull

GitHub, Inc. [US] | <https://github.com/critoma/javase>

critoma / javase

Java SE Lectures and Labs <http://acs.ase.ro>

15 commits 1 branch 0 releases 1 contributor MIT

Commit	Message	Time
critoma	Adding more details about NIO samples both lecture and labs for socke...	5 days ago
labs	Adding more details about NIO samples both lecture and labs for socke...	5 days ago
lectures	Adding more details about NIO samples both lecture and labs for socke...	5 days ago
LICENSE	Initial commit	8 months ago
README.md	Update README.md	8 days ago

README.md

javase

Java SE Samples for the Lectures and Labs source code with MIT License.



Questions & Answers!





Questions & Answers!

← → ↻ shell.cloud.google.com/?show=ide%2Cterminal&authuser=3&fromcloudshell=true

Apps ★ Bookmarks Mozilla Firefox Oracle / Sun Java / Android / O... Java / Kotlin / And... Python JS - N

Cloud Shell Editor

File Edit Selection View Go Run Terminal Help

EXPLORER

- SECITC
 - dad
 - ecdsapy
 - eth
 - ipctests
 - javase
 - c01
 - eu
 - ase
 - Hello.class
 - Hello.java**

API

JAVA DEPENDENCIES

- (default package)
 - Hello.java

1 package eu.ase;
2
3 public class Hello
4 {
5
6 Run | Debug
7 public static void main(String[] argv)
8 {
9 while(true)
10 {
11 System.out.println("My first Java");
12 }
13 }
14 }

2 14 Cloud Code minikube

cloudshell × + ▾

```
Welcome to Cloud Shell! Type "help" to get started.
To set your Cloud Platform project in this session use "gcloud config set project [PROJECT_ID]"
secitc@cloudshell:~$ cd javase/
secitc@cloudshell:~/javase$ cd c01
secitc@cloudshell:~/javase/c01$ javac eu/ase/Hello.java
secitc@cloudshell:~/javase/c01$ java eu.ase.Hello
```

Debug Project Explorer

ProgMainCertificate [Java Application]
c01.ProgMainCertificate at localhost:61762
Thread [main] (Suspended)
ProgMainCertificate.main(String[]) line: 29
D:\PROGRAMS\eclipse\plugins\org.eclipse.justi.openjdk.hotspot

Certificate.java ProgMainCertificate.java

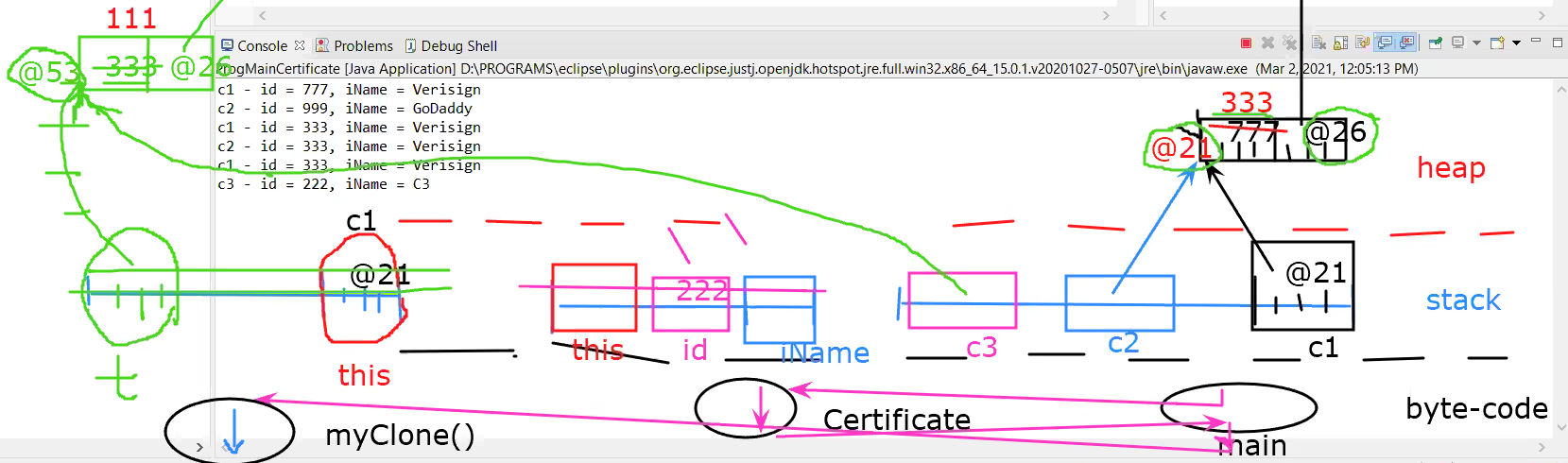
```
14 c2 = c1;  
15 //c2 = c1.myClone();  
16 c2.setIdCert(333);  
17  
18 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
19 System.out.println("c2 - id = " + c2.getIdCert() + ", iName = " + c2.getIssuerName());  
20  
21  
22 Certificate c3 = new Certificate(222, "C3");  
23 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
24 System.out.println("c3 - id = " + c3.getIdCert() + ", iName = " + c3.getIssuerName());  
25  
26 c3 = c1.myClone();  
27 c3.setIdCert(111);  
28  
29 System.out.println("c1 - id = " + c1.getIdCert() + ", iName = " + c1.getIssuerName());  
30 System.out.println("c3 - id = " + c3.getIdCert() + ", iName = " + c3.getIssuerName());  
31  
32 }  
33 }
```

Variables

Name	Value
setIdCert()	(No explicit return value)
args	String[] (id=20)
c1	Certificate (id=21)
idCert	333
issuerName	"Verisign" (id=26)
c2	Certificate (id=21)
c3	Certificate (id=53)
idCert	111
issuerName	"Verisign" (id=26)

<Choose a previously entered expression>

@26 2. @32 len value





Thanks!



Java Application Development
End of Lecture 1 – summary of Java SE

